



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

規格嚴格



功夫到家

1920 — 2017

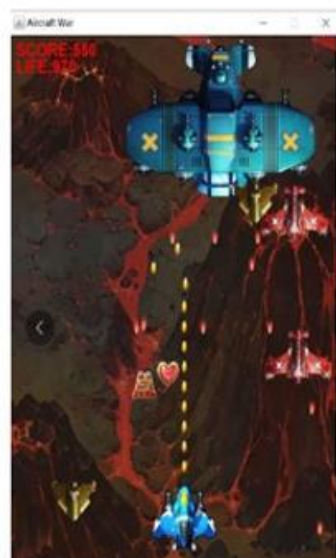
软件构造实践

Android开发技术

2026春

哈尔滨工业大学（深圳）

《软件构造》



- Java语言
- 面向对象思想
- 设计模式
- 多线程、泛型
- Swing、网络编程
- PC客户端飞机大战



《软件构造实践》



- Android飞机大战APP
- Android平台架构
- Android开发技术基础
- Android开发实践
- 调试、打包、发布等



本学期实验总体安排

实验项目	一	二	三	四	五		六	
学时	2	4	2	2	4	2	6	2
实验内容	Android 开发技术	代码迁移	界面	音效	排行榜	中期检查	网络功能开发	结题验收
提交内容	任务书							结题报告

说明：本课程是实践类课程，以学生实践为主，开发功能不受以上内容约束，可在在此基础上开发个人感兴趣内容。

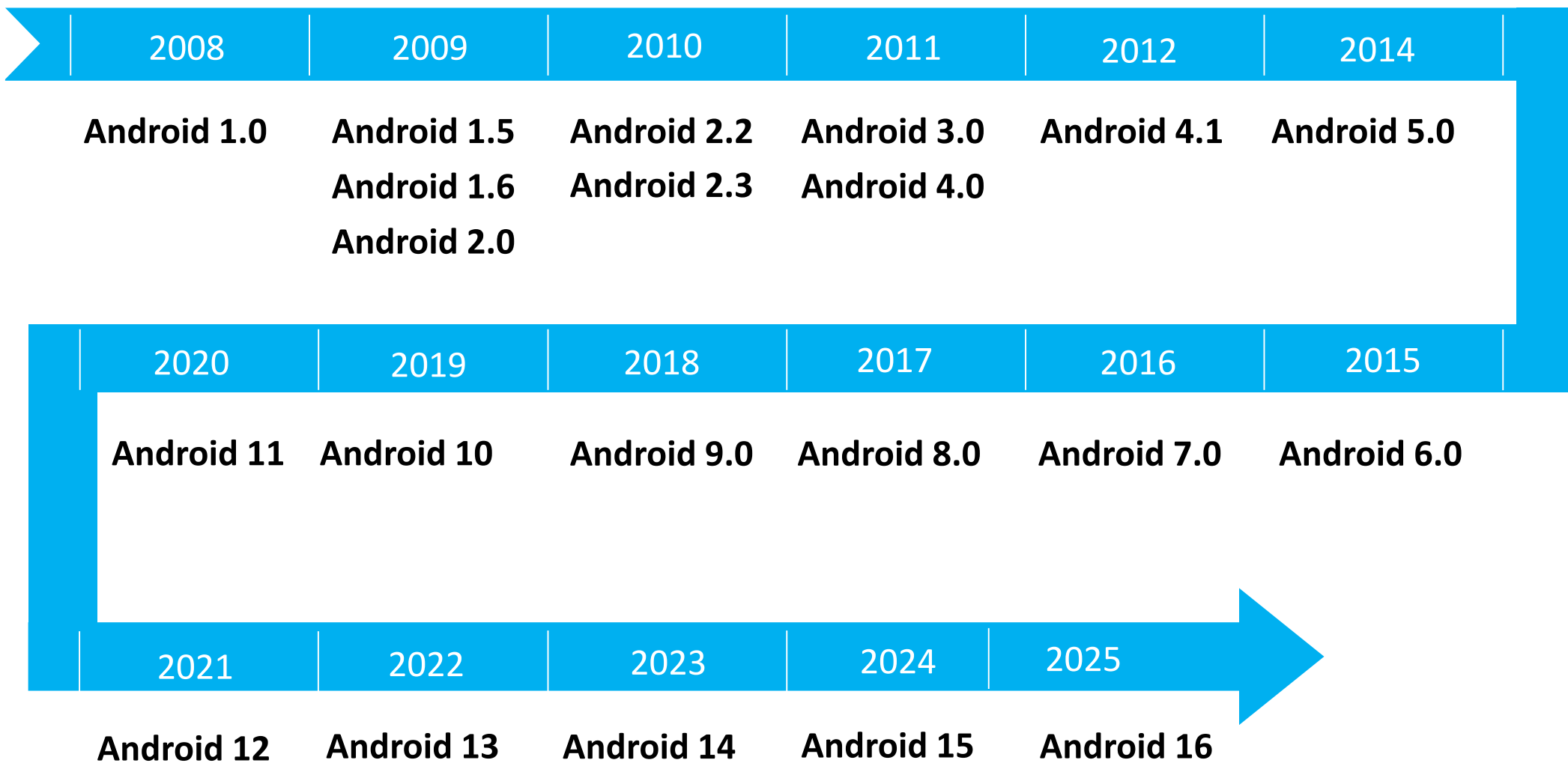


一、Android平台架构



二、Android应用开发技术

Android 系统版本

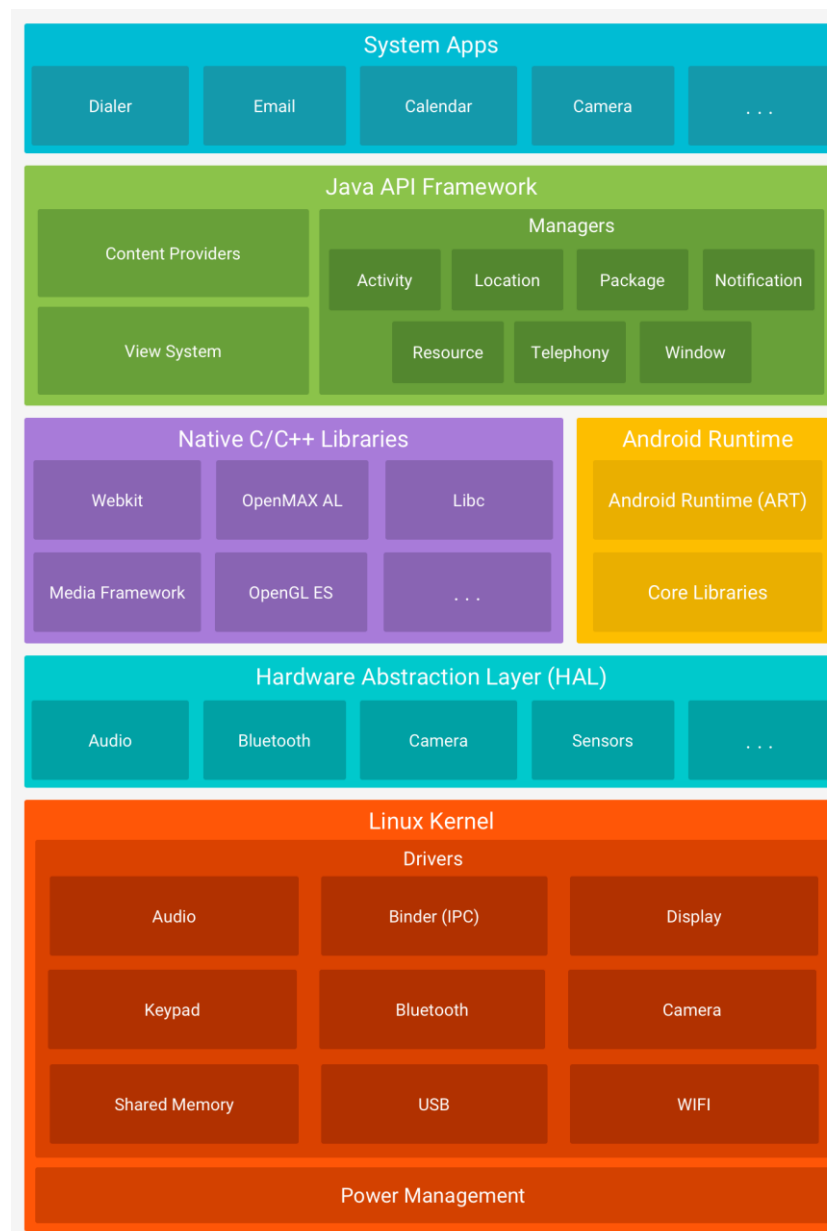


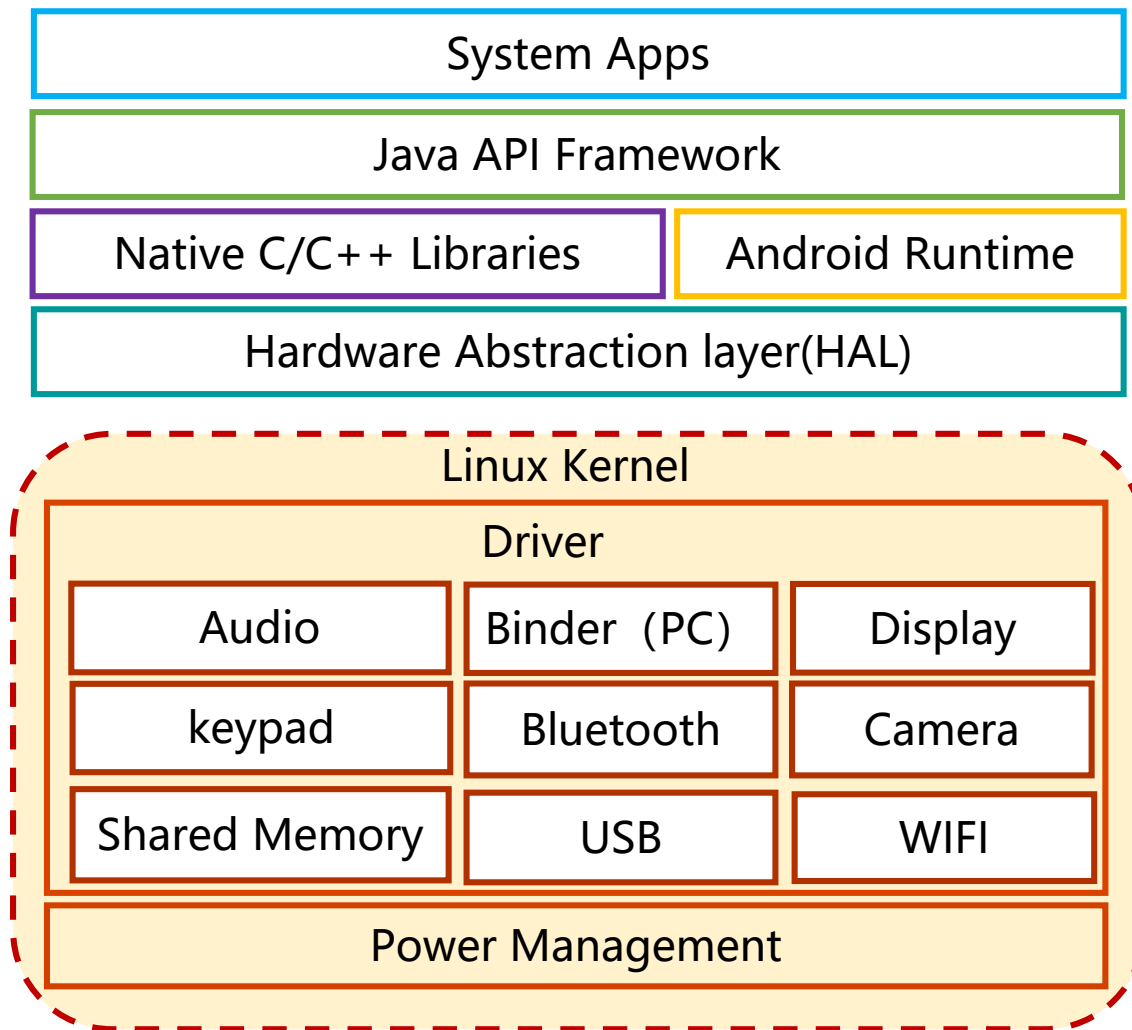
Android API级别

- 唯一标识Android 平台的版本及其框架 API的状态
- 控制应用安装的兼容性范围，如应用支持最低API级别和目标API级别
- 应用运行时，根据API决定功能的实现方式，避免应用在不同版本设备上因调用不存在的API而崩溃

Android平台	API级别
Android 5.0	21
Android 5.1	22
Android 6.0	23
Android 7.0	24
Android 7.1	25
Android 8.0	26
Android 8.1	27
Android 9	28
Android 10	29
Android 11	30
Android 12	31
Android 12L	32
Android 13	33
Android 14	34
Android 15	35
Android 16	36

Android平台架构



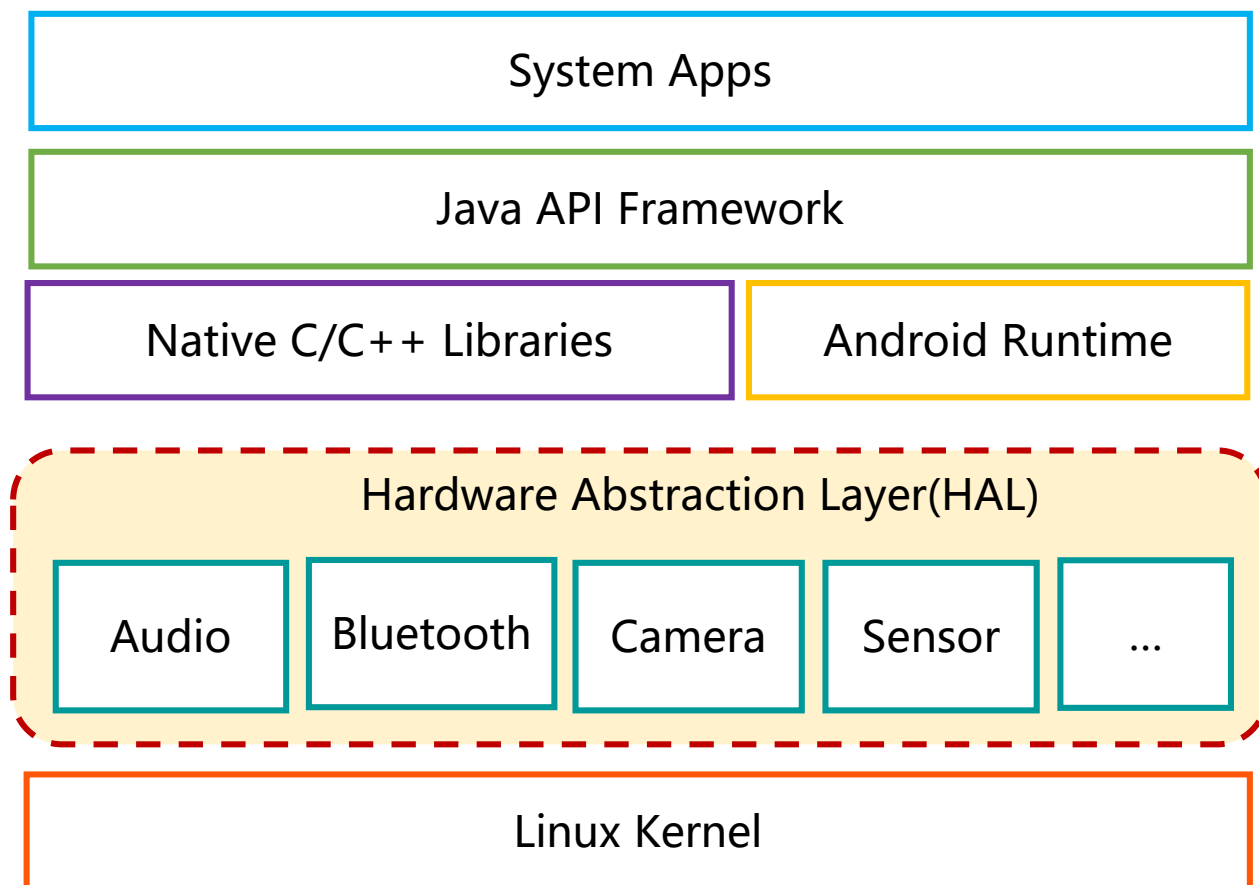


Android系统是Google开发的基于Linux平台的**开源**手机操作系统。

1.Linux内核

通过借助Linux内核服务实现一些核心功能：

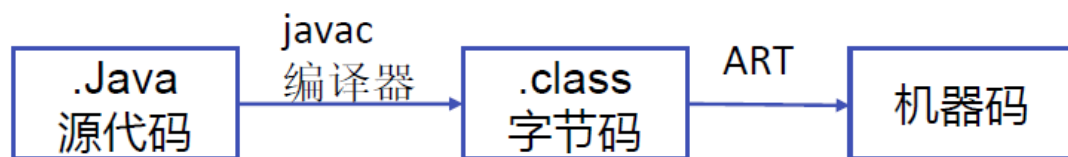
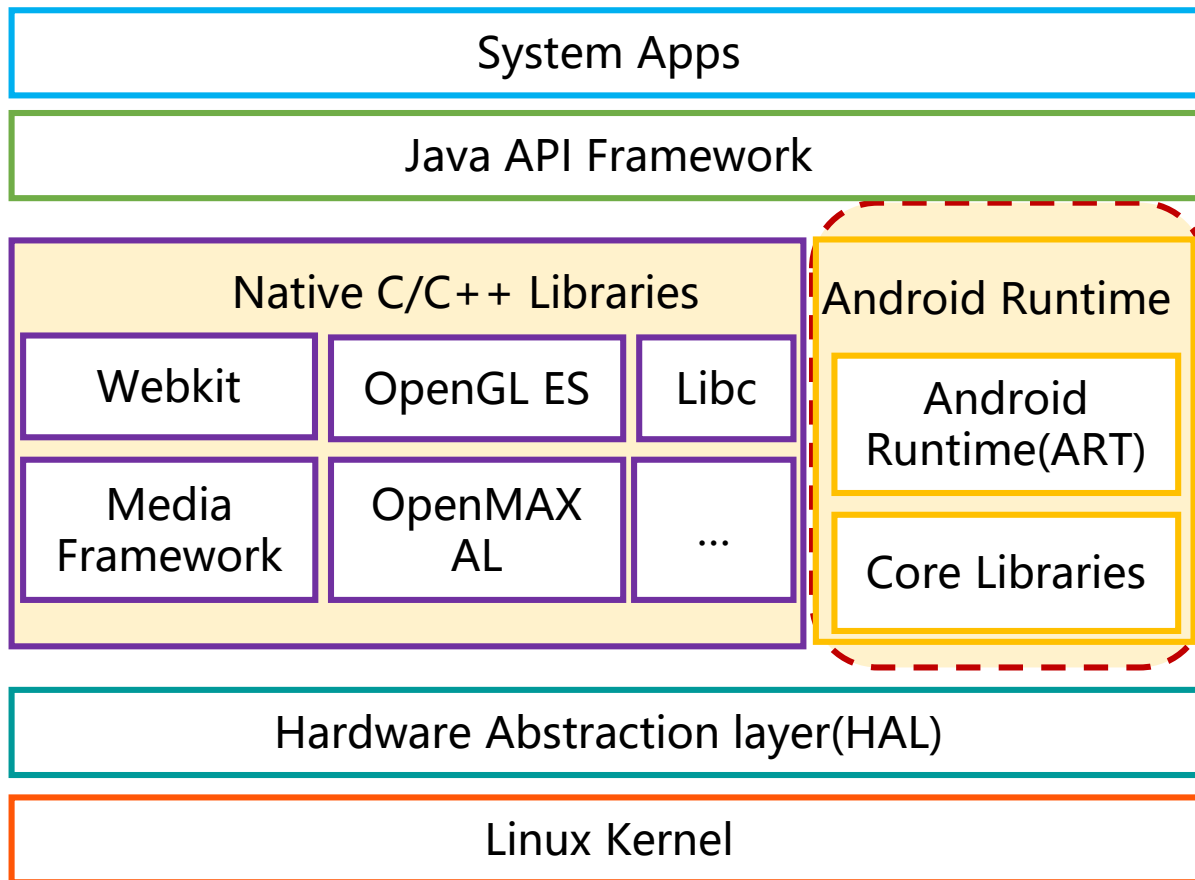
- 硬件设备驱动
- 进程和内存管理
- 网络协议栈
- 电源管理
- 无线通信等



2.硬件抽象层(HAL)

对Linux内核驱动程序的封装，将硬件抽象化，屏蔽掉了底层的实现细节。

HAL规定了一套应用层对硬件层读写和配置的统一接口。

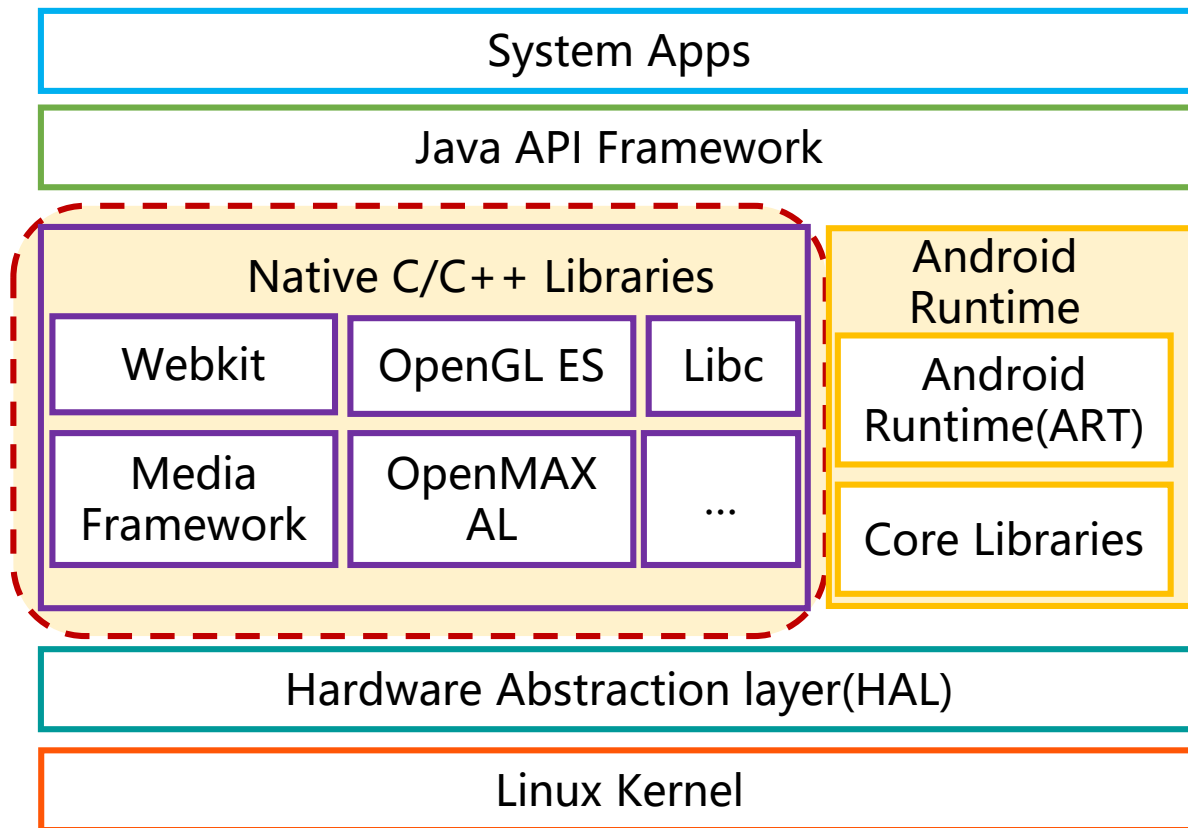


3.Android Runtime

AndroidRuntime (缩写为ART) , 是一种在Android系统上的运行环境。

- Android应用层和Framework是Java, Java代码被编译打包成字节码的dex或者odex文件;
- 而操作系统运行时需要的是机器码。

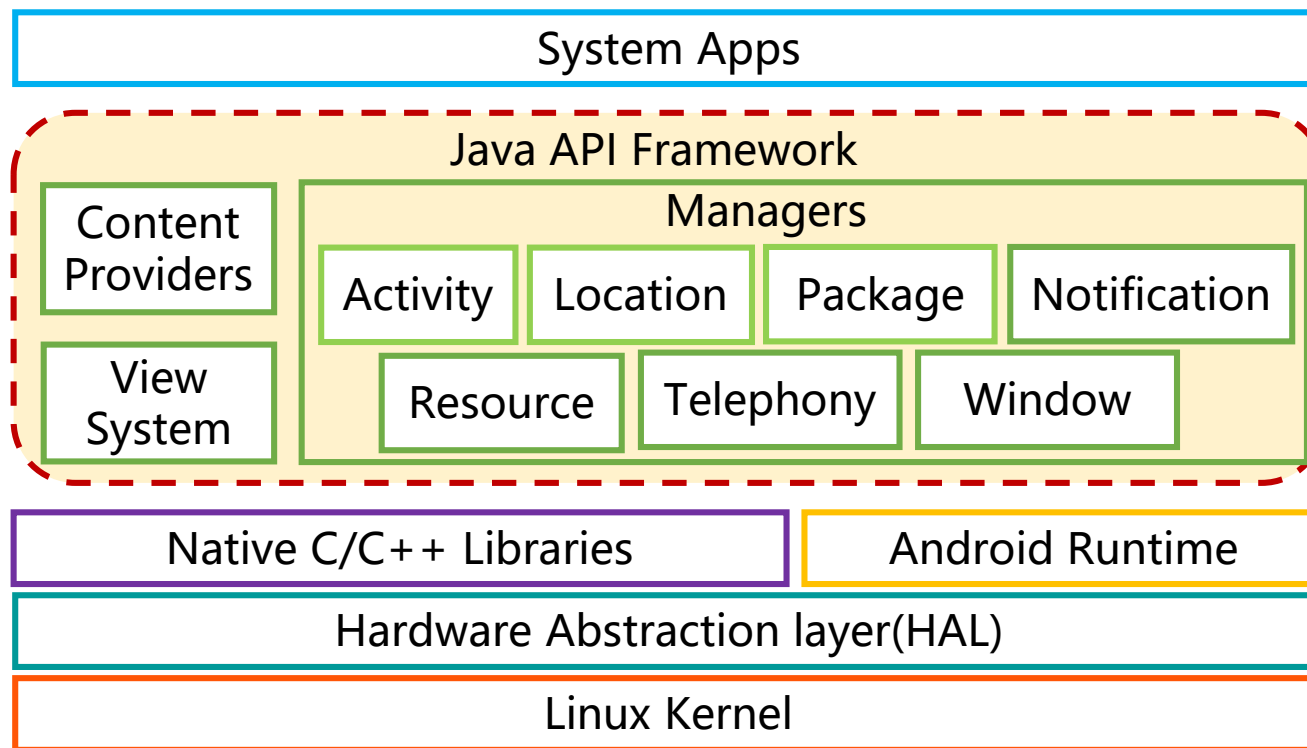
作用: Java代码转换成CPU处理器能够理解的机器码



4.原生C/C++库

✓ Android系统类库大部分由C/C++编写
✓ 通过JavaFramework层以JavaAPI的形式向应用层显示其中部分原生库的功能。

- Webkit: 浏览器的软件引擎
- OpenGL: 开放式图形库, 是用于渲染2D、3D矢量图形的跨语言、跨平台的应用程序编程接口
- Media Framework: 多媒体库, 支持多种常用的音频和视频格式的录制和回放



5.Java API FrameWork

由Java语言编写，提供了Android系统整个功能集的API集合；

例如：

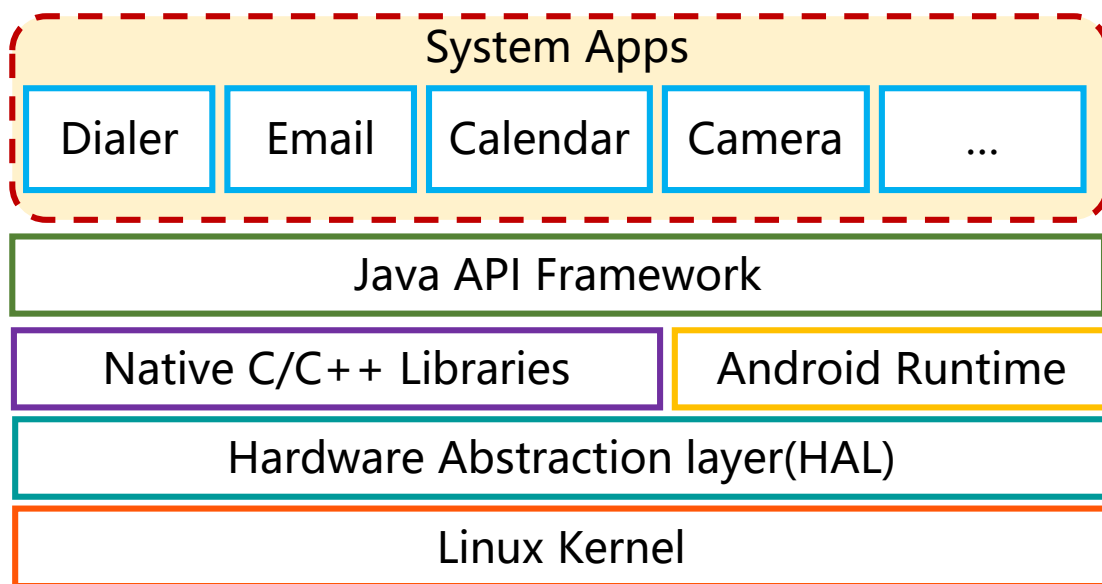
内容提供程序

视图系统

资源管理器

通知管理器

Activity管理等；



6.系统应用

- ①各类与用户直接交互的应用程序;
- ②运行于后台的服务程序。

电子邮件客户端

SMS程序

日历

地图

浏览器

联系人

其他设置等

了解Android系统架构分层的意义：

①从宏观上认识了Android系统，同时也给我们的学习与实践指明了方向；

A.若是从事Android应用开发，那应该研究Android的应用框架层和应用程序层；

B.若是从事Android系统开发，那应该研究Android的系统库和Android运行时。

C.若是从事Android驱动开发，那应该研究Android的Linux内核。

②采用分层架构的思想，架构清晰，层次分明，协同工作。



一、Android平台架构



二、Android应用开发技术

Android应用开发基本流程

1.设置工作区

安装AndroidStudio并创建项目。

2.编写应用

设计应用UI、交互、完成网络请求、数据处理等**业务逻辑编写**。

3.构建并运行

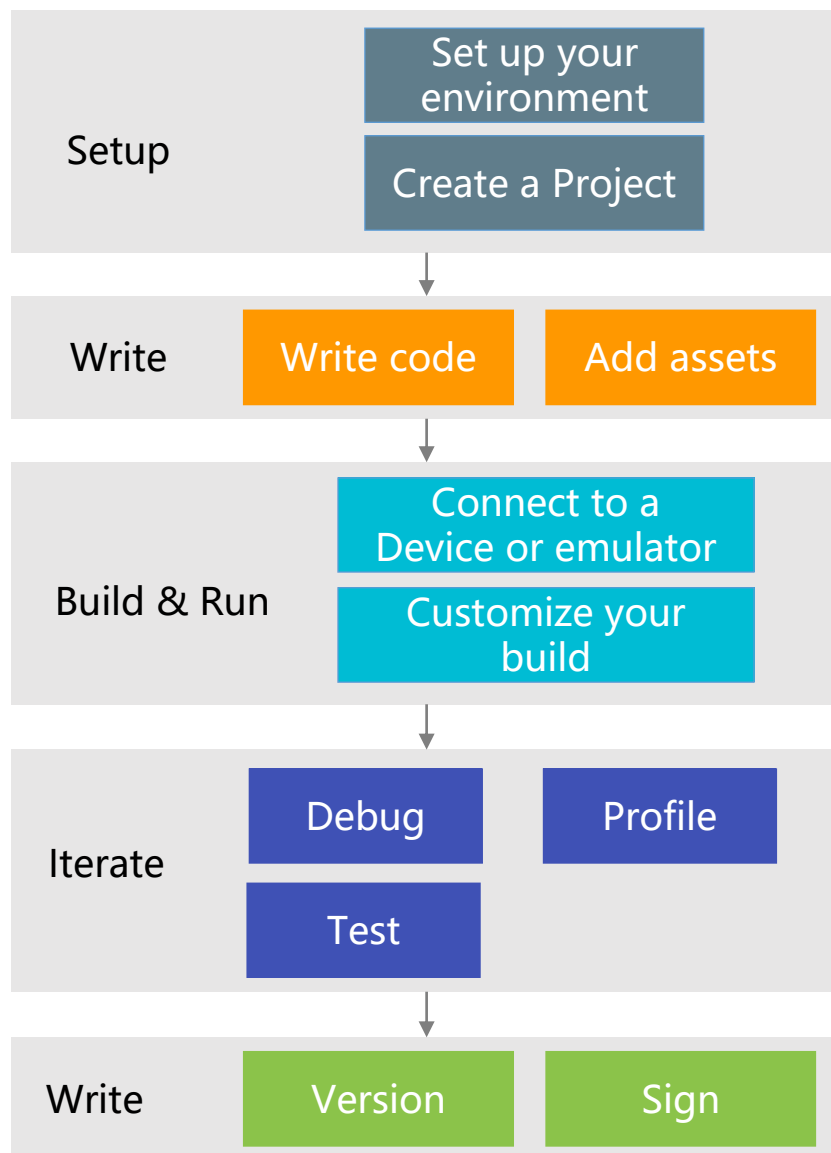
编译运行，将项目构建成一个可调试的APK软件包，以便在模拟器或Android设备上安装和运行。

4.调试、剖析和测试

消除错误并优化应用性能。查看和分析各种性能指标（如内存使用情况、网络流量、CPU占用等）。

5.发布

当准备向用户发布应用时，需要考虑管理应用版本、运营、构建APK包、使用密钥为应用签名。





1. 开发语言、开发环境
2. UI(User Interface)开发
3. 交互
4. 进程、线程
5. 网络和数据
6. 架构模式
7. 调试分析
8. 打包发布



IntelliJ IDEA: Java集成开发环境, 安装插件后可支持Android开发。



Android Studio: 是谷歌专门为Android “量身定做” 的集成开发环境。

推荐使用

- 零配置、即装即用;
- 增强版的布局编辑器;
- 更完善的调试系统;
- 默认集成gradle并优化构建流程。

- **JAVA**: Android开发的传统语言, 生态成熟, 大量的开源库和框架支持。
- **Kotlin**: Android开发的新选择, 语法简洁, 开发效率高, 可与Java互操作, 适配Android Studio新特性。

Android Studio版本:
Android Studio Narwhal (待确认)



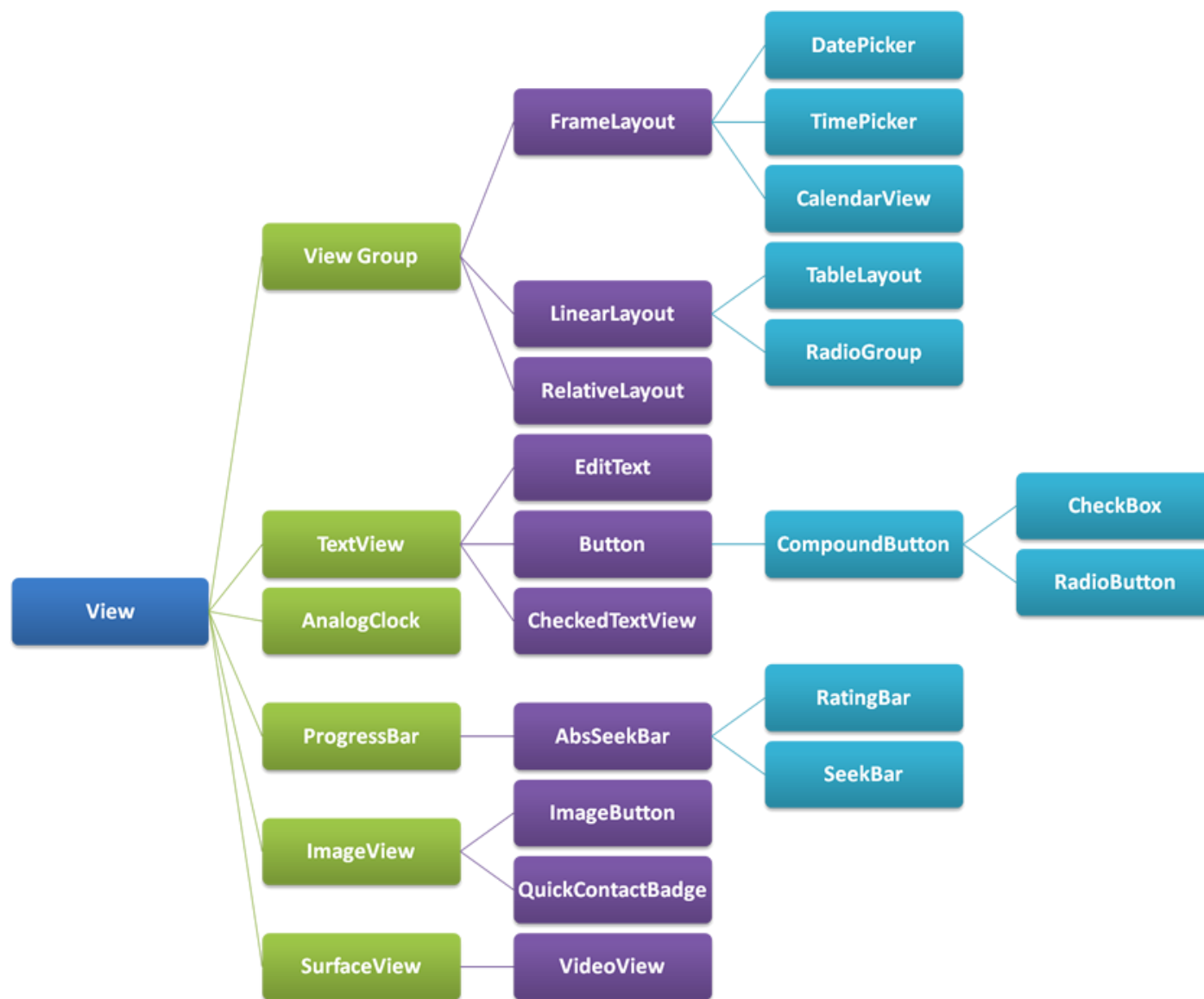
UI: User Interface

UI交互设计：是指对软件的人机交互、操作逻辑、界面美观的整体设计。

飞机大战UI元素：

- ① View: TextView、ImageView、Button...
- ② ViewGroup: LinearLayout
RelativeLayout
Constraint Layout
- ③ Activity、Fragment...

Android View



Android游戏开发常用的三种视图：View、SurfaceView和GLSurfaceView。

View

- 提供内置画布、图形绘制功能以及处理用户输入事件的能力
- 在UI主线程内更新画面

SurfaceView

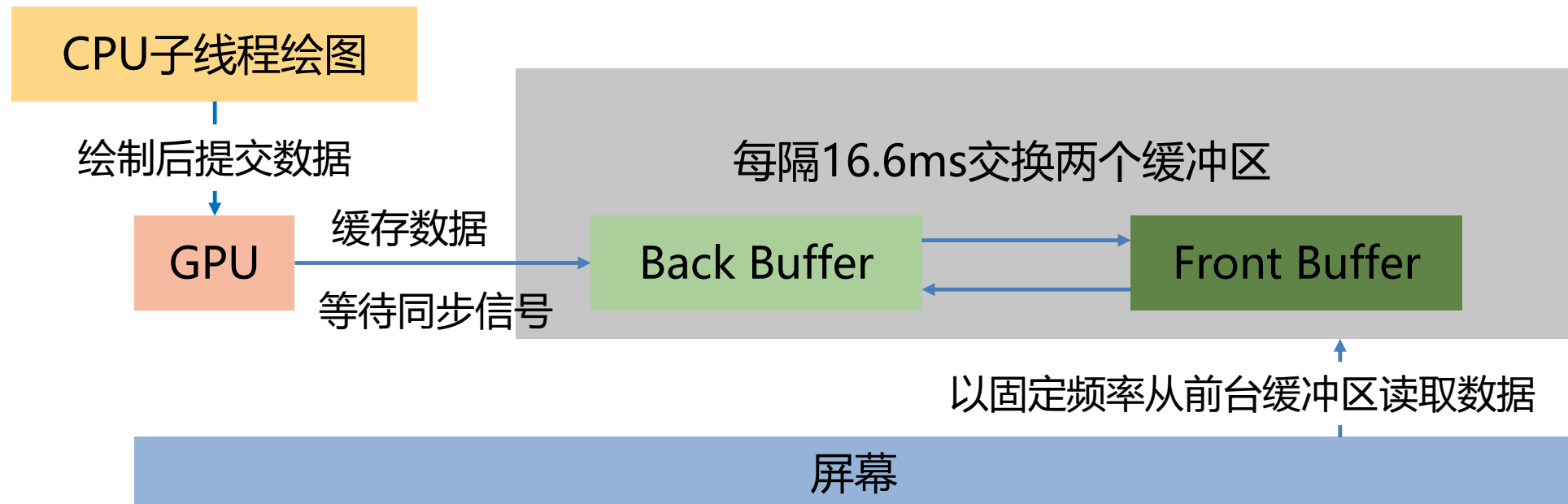
- View类的子类
- 拥有独立的Surface和缓冲队列
- 在新的子线程中更新画面，不阻塞主线程
- 适用于2D游戏的开发

GLSurfaceView

- SurfaceView的子类
- 封装了OpenGL渲染
- 适用于3D绘图场景

UI的主线程中更新画面可能会引发问题：

假如更新画面的时间过长，导致主UI线程会被正在画的函数阻塞，从而无法响应按键，触屏等消息。



□双缓冲优点

- ①流畅，绘制过程中不会出现闪烁花屏现象；
- ②高效，将图像一次性绘制到屏幕上比一次绘制一部分要高效的多。

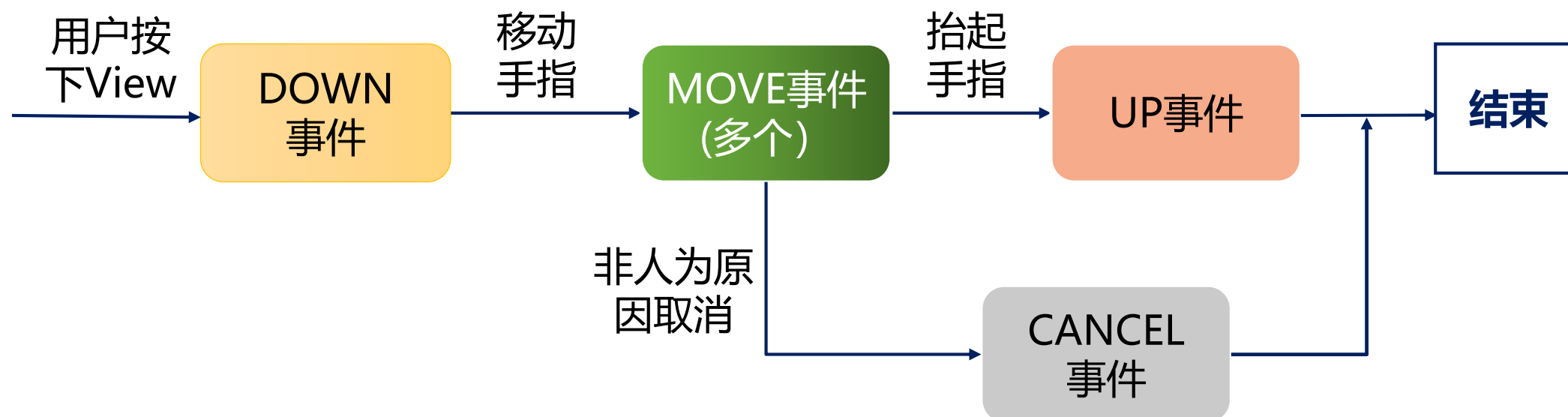
□双缓冲缺点

图片过大时会严重消耗内存。



- ①按钮点击;
- ②手势、移动鼠标、点击屏幕;
- ③点击系统Home键、点击系统Back键.....

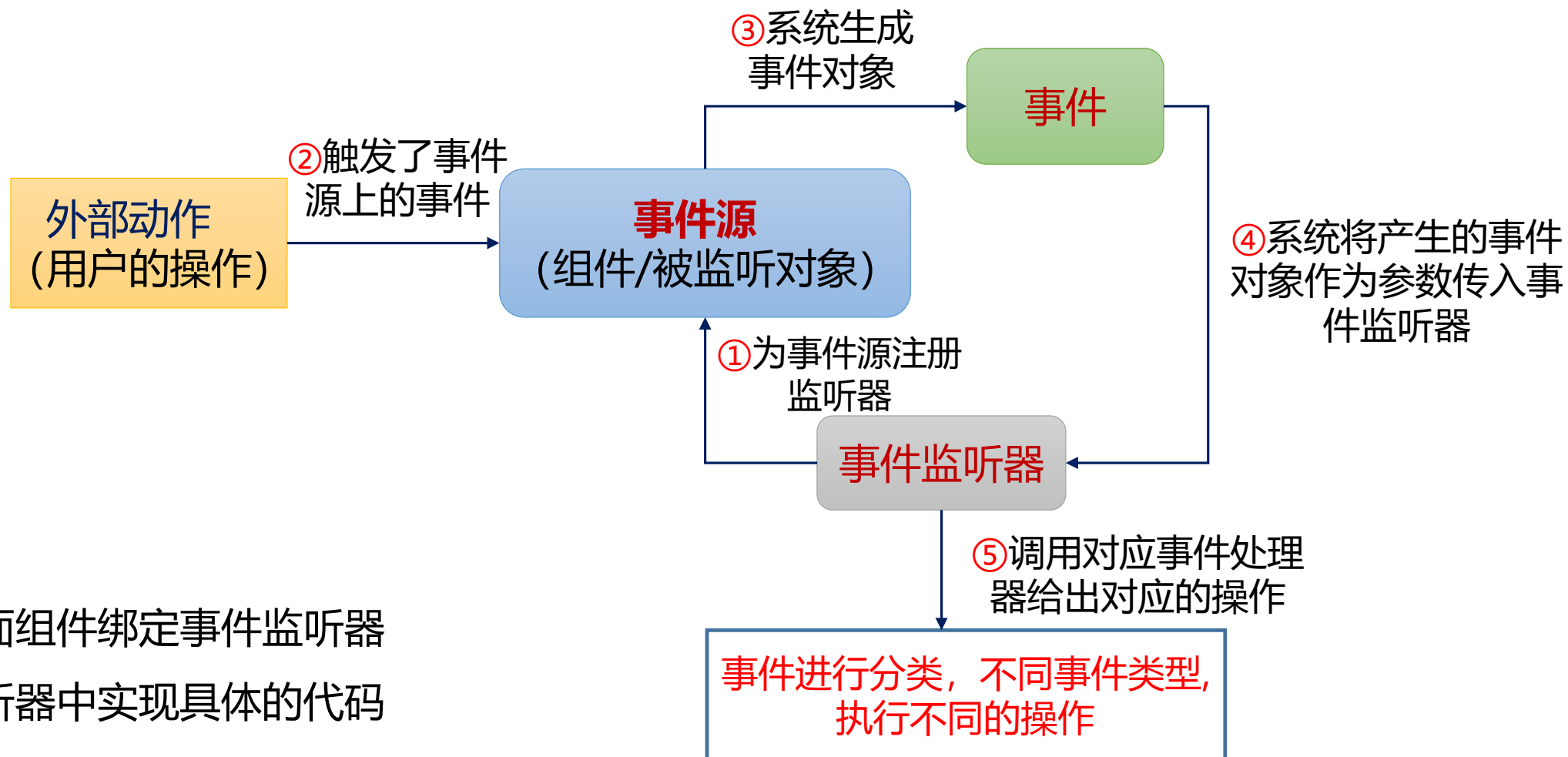
.....



□Android两种事件处理模型

- ①基于监听的事件处理
- ②基于回调的事件处理

流程模型图



总结：

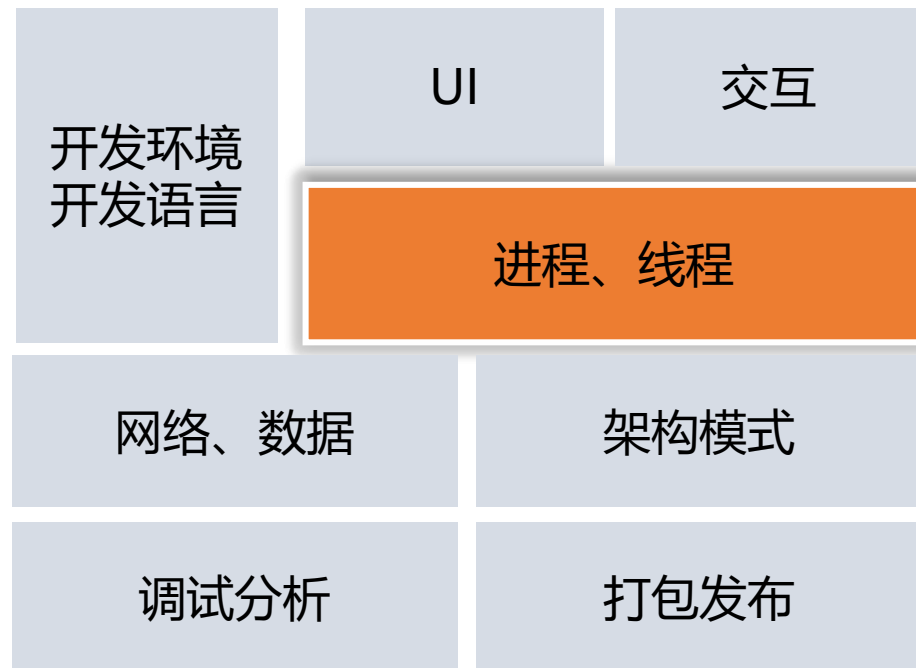
- ① 给界面组件绑定事件监听器
- ② 在监听器中实现具体的代码

- ① 无事件监听器
- ② 用户在组件上触发某个事件时，**组件自己特定的方法**负责处理该事件
- ③ 为了实现回调机制的事件处理，Android为所有的GUI组件都提供了回调方法
- ④ 以View为例来说明，可以通过重写View中的这些回调方法来实现需要的响应事件

应用场景：自定义View控件

- *boolean onKeyDown(int keyCode,KeyEvent event);用户在该组件上**按下键盘时**触发的
- *boolean onKeyLongPress(int keyCode,KeyEvent event);用户在该组件上**长按按键时**触发的
- *boolean onKeyShortcut(int keyCode,KeyEvent event);当一个键盘**快捷键事件**发生时触发该方法
- *boolean onKeyUp(int keyCode,KeyEvent event);用户**松开**按键的时候触发的事件
- *boolean onTouchEvent(MotionEvent event);用户在该组件上触发**触摸屏时**触发该方法

进程、线程



任务管理器

文件(F) 选项(O) 查看(V)

进程 性能 应用历史记录 启动 用户 详细信息 服务

名称	状态	6% CPU	59% 内存	1% 磁盘	0% 网络
应用 (11)					
> Android Studio		0%	202.9 MB	0 MB/秒	0 Mbps
> Google Chrome (42)		0.3%	1,198.2 ...	0.1 MB/秒	0.1 Mbps
> Microsoft Edge (10)		0%	39.0 MB	0 MB/秒	0 Mbps
> Microsoft PowerPoint (3)		0%	198.6 MB	0 MB/秒	0 Mbps
> Microsoft Word		0%	26.4 MB	0 MB/秒	0 Mbps
> Notepad++ : a free (GNU) sour...		0%	0.7 MB	0 MB/秒	0 Mbps
> WeChat (32 位) (8)		0%	168.4 MB	0 MB/秒	0 Mbps
> Windows 资源管理器 (3)		0.2%	69.5 MB	0 MB/秒	0 Mbps
> 电脑管家 (32 位)		0%	3.2 MB	0 MB/秒	0 Mbps
> 任务管理器		0.5%	32.5 MB	0 MB/秒	0 Mbps
> 有度即时通 (32 位)		0.8%	30.9 MB	0 MB/秒	0 Mbps

Android应用进程

```
generic_x86:/ $ ps -ef | grep com
root      36      2 0 05:38:36 ?      00:00:00 [kcompactd0]
root      99      2 0 05:38:36 ?      00:00:00 [krfcomm]
system    307      1 2 05:38:55 ?      00:01:45 android.hardware.graphics.composer@
bluetooth 682     289 0 05:39:07 ?      00:00:11 com.android.bluetooth
u0_a112   706     289 0 05:39:07 ?      00:00:32 com.android.systemui
network_stack 842    289 0 05:39:08 ?      00:00:03 com.android.networkstack.process
secure_element 880    289 0 05:39:08 ?      00:00:00 com.android.se
radio     914     289 0 05:39:08 ?      00:00:26 com.android.phone
u0_a107   1060    289 0 05:39:09 ?      00:00:12 com.android.launcher3
u0_a87    1176    289 0 05:39:10 ?      00:00:17 com.google.android.gms.persistent
u0_a103   1265    289 0 05:39:11 ?      00:00:00 com.android.inputmethod.latin
u0_a117   1341    289 0 05:39:12 ?      00:00:02 com.android.providers.media.module
u0_a87    1369    289 0 05:39:12 ?      00:00:10 com.google.android.gms
radio     1399    289 0 05:39:12 ?      00:00:00 com.android.ims.rcsservice
system    1428    289 0 05:39:12 ?      00:00:00 com.android.emulator.multidisplay
u0_a87    1525    289 0 05:39:12 ?      00:00:01 com.google.process.gservices
system    6575    289 0 07:12:20 ?      00:00:00 com.android.keychain
u0_a87    6612    289 0 07:12:20 ?      00:00:00 com.google.android.gms.ui
u0_a114   6669    289 0 07:12:21 ?      00:00:00 com.android.permissioncontroller
u0_a123   6860      1 0 07:14:45 ?      00:00:00 install_server-6a9a542f com.hitsz
u0_a123   6879    289 0 07:14:46 ?      00:00:04 com.hitsz
u0_a87    7342    289 0 07:27:08 ?      00:00:00 com.google.android.gms.unstable
u0_a124   7448    289 30 07:31:29 ?      00:00:06 com.tencent.qqlive:services
u0_a124   7502    289 12 07:31:32 ?      00:00:02 com.tencent.qqlive:pmervice
u0_a124   7553    289 11 07:31:32 ?      00:00:01 com.tencent.qqlive:xg_vip_service
u0_a124   7706    289 22 07:31:33 ?      00:00:03 com.tencent.qqlive:cache
```

□ 进程的启动:

启动App, Android会启动一个Linux进程和一个主线程。

□ 进程的退出:

①被动结束进程: 根据进程中运行的组件类别以及组件的状态来判断该进程的优先级, Android会首先停止那些优先级低的进程。

②主动结束进程: 主动结束程序运行。

□ 进程优先级:

前台进程 > 可见进程 > 服务进程 > 后台进程 > 空进程

□ Android线程

主线程：进程拥有的默认线程，即**UI线程**；

子线程：进程中手动创建的线程，用于执行耗时任务。

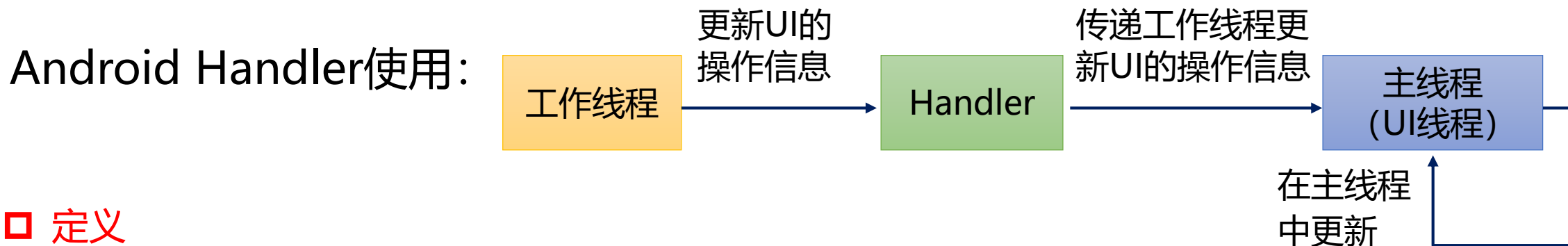
□ 多线程业务场景

网络数据交互：访问网络获取商品列表，

用户信息校验

耗时的操作：图片处理、音视频解码、复杂的数学计算





□ 定义

一套Android消息传递机制。

□ 作用

在多线程的应用中将**工作线程**中需更新UI的操作信息**传递到UI主线程**，实现主线程对UI的更新处理，最终实现异步消息的处理。

UI线程阻塞超过5s会发生ANR (Application NotResponse) 错误。

□ 解决问题

- Android的UI访问是没有加锁的，所以多个线程访问UI是不安全的
- 规定只能在UI线程中访问UI，故而主线程的响应速度不应该受到影响
- **耗时操作**应该放置到**子线程**中进行


```
// 采用继承Thread类实现多线程演示
new Thread() {
    @Override
    public void run() {
        try {
            Thread.sleep( millis: 4000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        // 步骤3: 创建所需的消息对象
        Message msg = Message.obtain();
        msg.what = 1; // 消息标识
        msg.obj = "A"; // 消息内存存放

        // 步骤4: 在工作线程中 通过Handler发送消息到消息队列中
        mHandler.sendMessage(msg);
    }
}.start();
```

```
// 步骤1: (自定义) 新创建Handler子类(继承Handler类) & 复写handleMessage()
class Mhandler extends Handler {

    // 通过复写handleMessage() 从而确定更新UI的操作
    @Override
    public void handleMessage(Message msg) {
        // 根据不同线程发送过来的消息, 执行不同的UI操作
        // 根据 Message对象的what属性 标识不同的消息
        switch (msg.what) {
            case 1:
                mTextView.setText("执行了线程1的UI操作");
                break;
            case 2:
                mTextView.setText("执行了线程2的UI操作");
                break;
        }
    }
}
```

□什么是Service

Service是Android中实现程序**后台运行**的组件，适用于执行不需要和用户交互而且长期运行的任务，默认**运行在主线程**。

□Service的适用场景

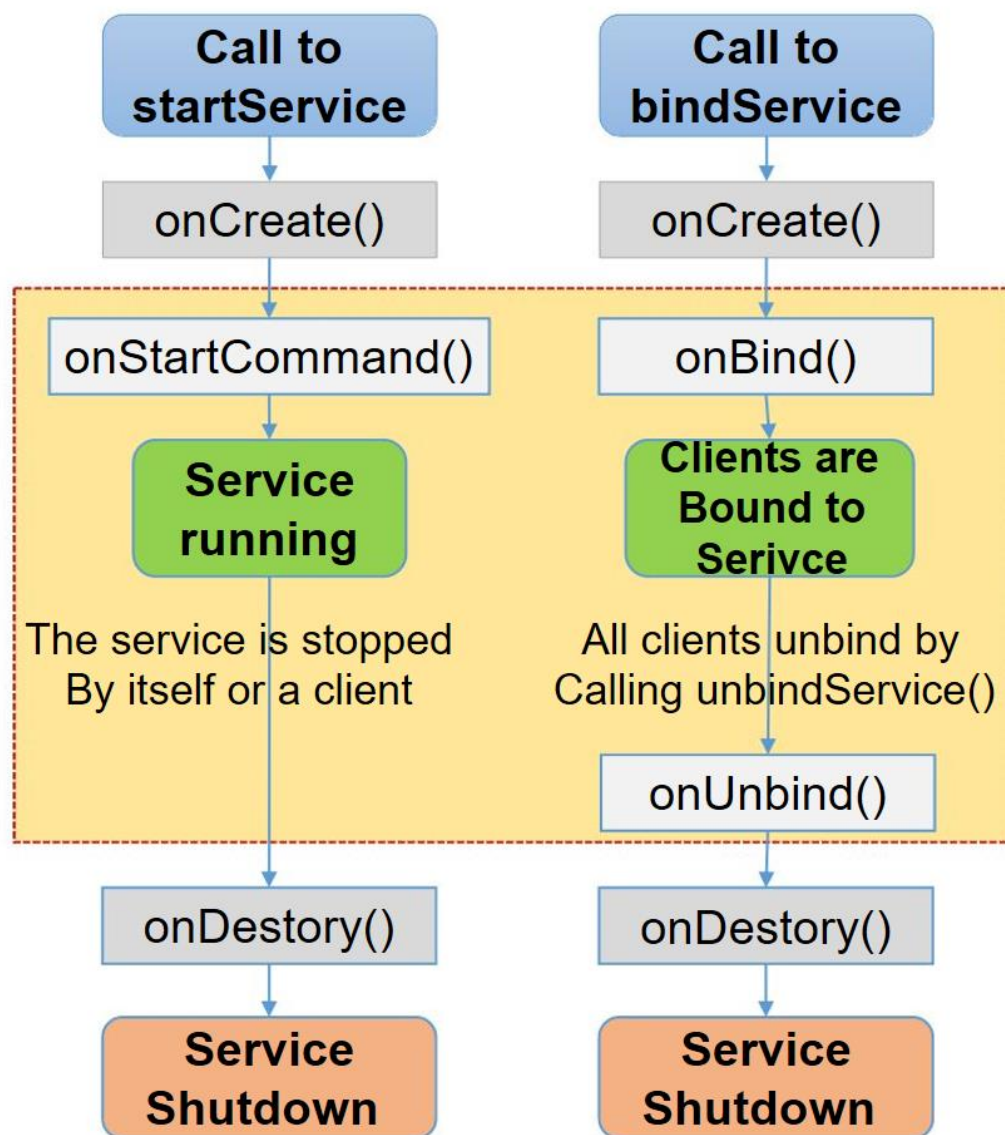
不依赖于用户可视的UI界面
具有较长时间的运行特性

□飞机大战游戏Service使用场景

背景音乐播放

不同	Thread	Service
概念	程序执行的最小单位，分配CPU的基本单位	没有UI的Android后台组件
执行任务	可执行耗时任务	运行在主线程，管理后台任务；如果需要执行耗时任务，在内部启动Thread
生命周期	由开发者控制	生命周期由Android系统管控

Service的生命周期



Service使用方法:

- ① 新建TestService继承Service,重写父类的回调方法
- ② 在 AndroidManifest 中 配置 Service (Android Studio新版本新建Service类时会自动添加)
- ③ 执行Service

类型	特点	应用场景
启动服务	一旦启动, 即使启动它的组件 (如 Activity) 关闭, 服务仍继续运行, 直到完成任务	通常用于不依赖界面的长期任务
绑定服务	与调用它的组件 (如 Activity) 绑定在一起, 组件销毁, 服务也随之销毁	适合需要与 UI 协作的任务



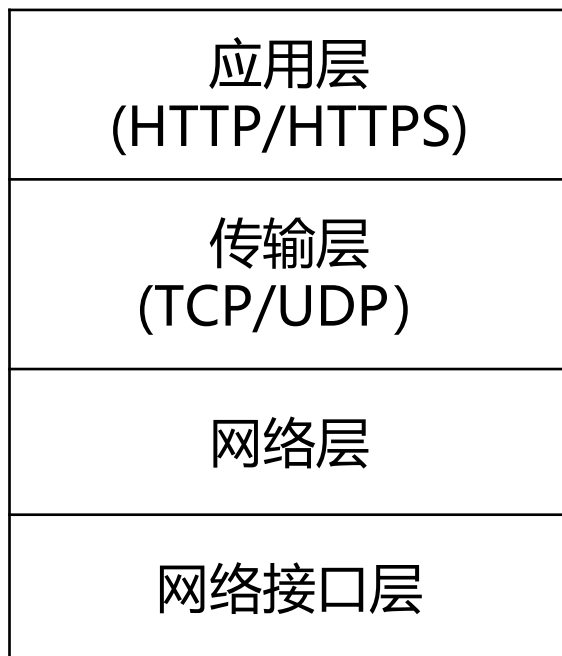
网络业务场景：

- 账户登录时用户信息服务端校验
- 游戏得分云同步、双人对战得分实时显示
- 商品道具列表

数据存储业务场景：

- 游戏模式设置：本次选择的游戏模式在下次启动时依然有效
- 单机版得分排行榜记录
- 应用的配置信息

网络模型



Android 网络技术

HttpURLConnection
OKHttp

Socket

```
<?xml version="1.0" encoding="utf-8"?>
<manifest>
  <!-- 网络访问核心权限 -->
  <uses-permission android:name="android.permission.INTERNET" />
  <application
    android:label="@string/app_name"
    <!-- 允许 HTTP 明文流量（Android 9.0+ 必备） -->
    android:usesCleartextTraffic="true">
  </application>
</manifest>
```

- 主线程不能执行网络操作
- 必须在AndroidManifest.xml中声明**INTERNET**权限才能访问网络
- Android 9.0+ 默认禁止 HTTP 明文流量，强制 HTTPS；使用HTTP需修改AndroidManifest.xml

Android 原生API，可用于指定URL并发送GET请求、POST请求。

步骤1

- 获得HttpURLConnection实例，传入目标的网络地址

步骤2

- 设置HTTP请求参数：GET/POST请求、超时时间等

步骤3

- 调用connect()连接远程资源

步骤4

- 使用getOutputStream()发送数据给服务器
- 使用getInputStream()读取服务器的响应数据

步骤5

- 关闭HttpURLConnection连接

Square公司推出的一款开源的处理HTTP请求的轻量级框架。

- 在build.gradle中添加依赖

```
dependencies {  
    // OkHttp 核心依赖  
    implementation 'com.squareup.okhttp3:okhttp:4.12.0'  
}
```

- 创建OkHttp实例，构建Request，定义URL、方法等
- 发起异步请求

- 处理响应Response对象

① 失败处理: onFailure回调方法捕获异常

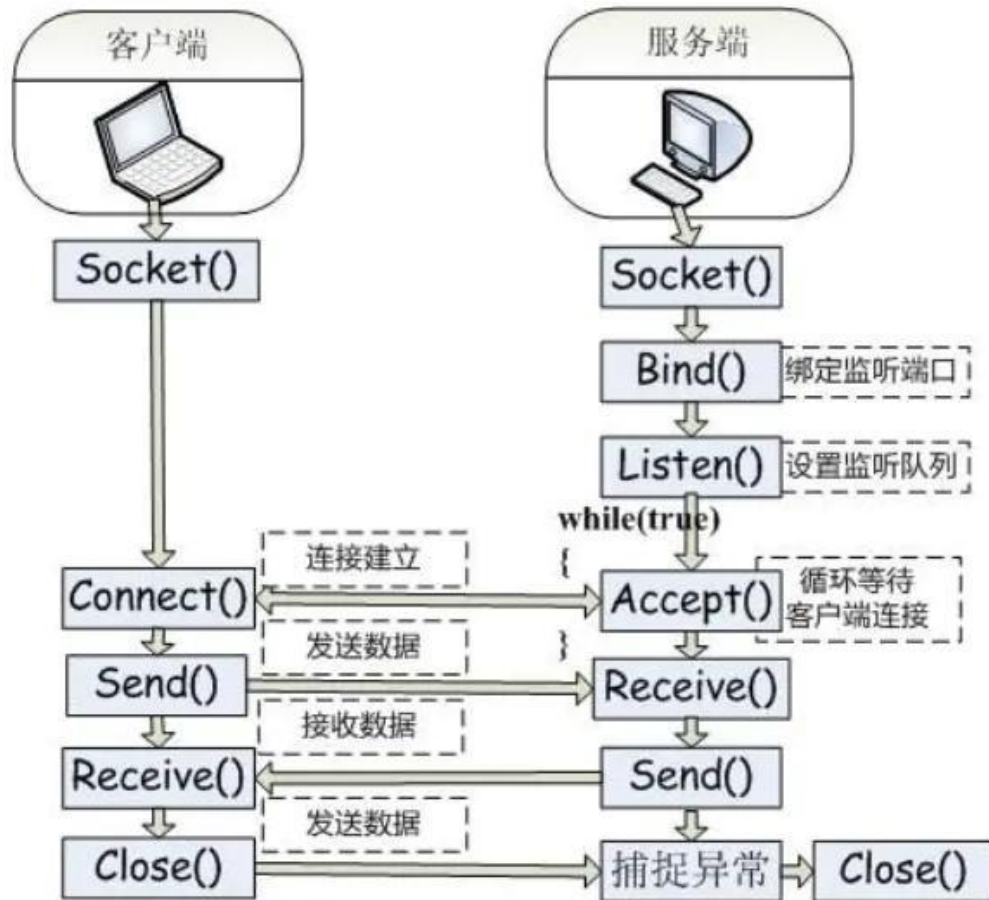
② 成功处理: onResponse回调方法判断状态码，解析响应体

```
// 1. 创建 OkHttpClient 实例  
OkHttpClient client = new OkHttpClient();  
  
// 2. 构建 GET 请求 (目标地址: 公开测试接口, 返回随机用户数据)  
Request request = new Request.Builder()  
    .url("https://randomuser.me/api/") // 测试用的公开 API  
    .get() // 默认就是 GET 方法, 可省略  
    .build();  
  
// 3. 发起异步请求 (非阻塞, 适合 Android 主线程)  
client.newCall(request).enqueue(new Callback() {  
    // 4. 失败处理 (网络异常、超时等)  
    @Override  
    public void onFailure(Call call, IOException e) {  
        e.printStackTrace();  
        System.out.println("请求失败: " + e.getMessage());  
    }  
  
    // 5. 成功处理 (解析响应)  
    @Override  
    public void onResponse(Call call, Response response) throws IOException {  
        if (response.isSuccessful()) { // 判断状态码是否为 200-299  
            // 获取响应体内容 (字符串格式)  
            String responseData = response.body().string();  
            System.out.println("请求成功, 响应数据: \n" + responseData);  
        } else {  
            System.out.println("请求失败, 状态码: " + response.code());  
        }  
    }  
});
```

□ OkHttp优势

- 自动线程管理: enqueue异步请求自动在子线程执行, 回调方法默认在子线程
- 代码量少、可读性高

- 基于Socket（套接字）实现网络通信
- Socket 建立连接后，客户端和服务端可以**同时发送和接收数据**，是“双向管道”模式，适合**实时性要求高**的场景



- ① 服务器实例化一个ServerSocket对象
- ② 服务器调用ServerSocket类的accept()方法，等待客户端的连接
- ③ 客户端尝试实例化一个Socket对象，指定服务器名称和端口号来请求连接
- ④ 连接建立，服务器获得一个新的Socket对象
- ⑤ 客户端和服务端使用Socket实例进行网络通信，使用InputStream和OutputStream来交换数据

1

SharedPreferences
存储

- 用**键值对**存储少量关键数据
- 适合保存配置、设置、用户偏好等数据
- 只限**内部私有存储**

2

文件
存储

- 保存**非结构化数据**
- 适合保存文本、音频、视频、图片等
- **任意位置存储均可**

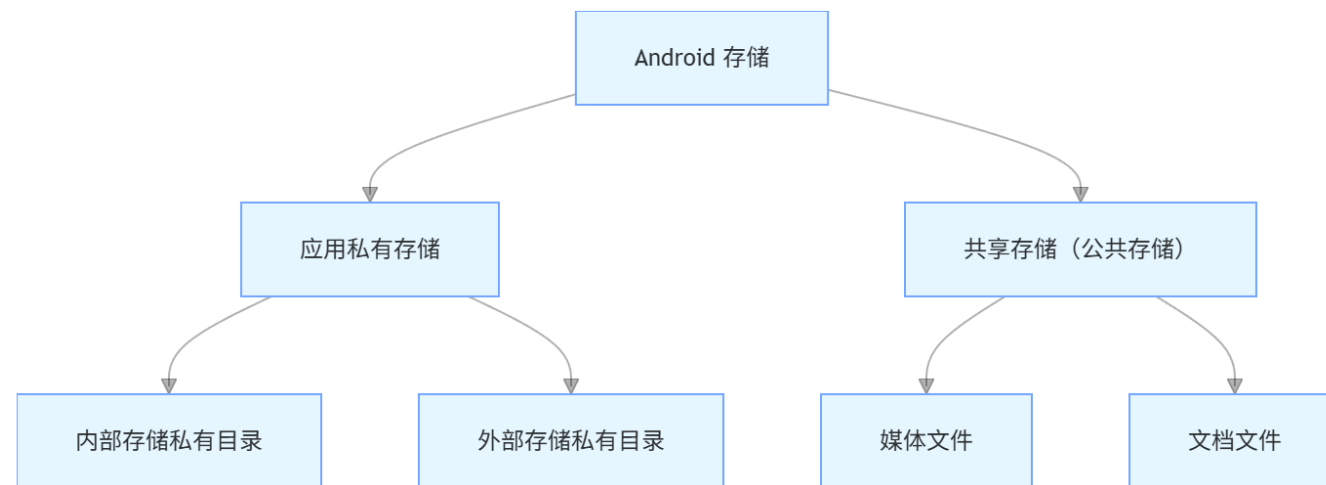
3

SQLite
数据库存储

- 保存**大量结构化数据**
- 系统自带，便于对数据进行CRUD操作
- 默认**内部私有存储**

View-Tool Windows-Device Explorer

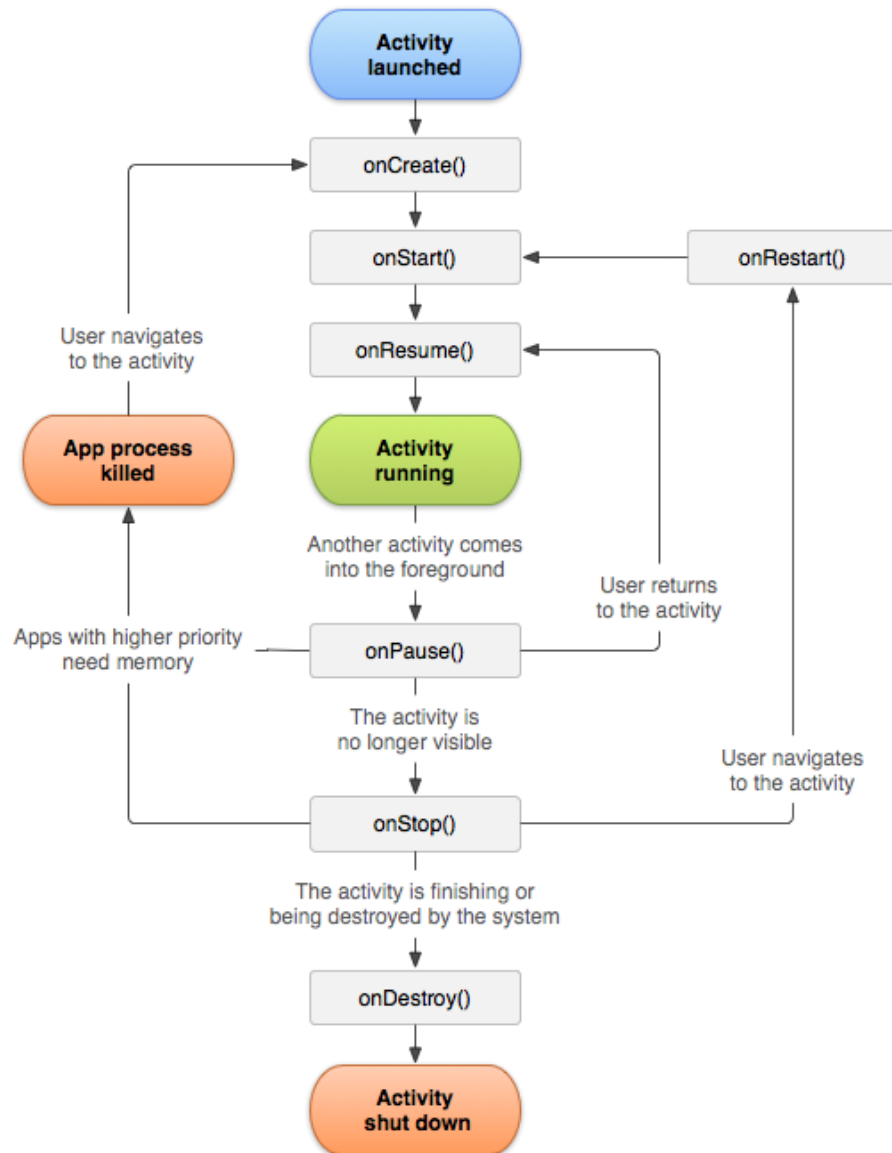
Name	Permissions	Date	Size
> data	drwxrwx--x	2024-10-23 07:50	4 KB
> debug_ramdisk	drwxr-xr-x	2009-01-01 00:00	4 KB
> dev	drwxr-xr-x	2024-10-23 07:49	2.6 KB
> etc	lrw-r--r--	2009-01-01 00:00	11 B
> lost+found	drwx-----	2009-01-01 00:00	16 KB
> mnt	drwxr-xr-x	2024-10-23 07:49	320 B
> odm	drwxr-xr-x	2009-01-01 00:00	4 KB
> odm_dilkm	drwxr-xr-x	2009-01-01 00:00	4 KB
> oem	drwxr-xr-x	2009-01-01 00:00	4 KB
> proc	dr-xr-xr-x	2024-10-23 07:49	0 B
> product	drwxr-xr-x	2009-01-01 00:00	4 KB
> sdcard	lrw-r--r--	2009-01-01 00:00	21 B
> second_stage_resources	drwxr-xr-x	2009-01-01 00:00	4 KB
▼ storage	drwx--x---	2024-10-23 07:50	100 B
> 0A0F-360D	drwxrwx---	1970-01-01 00:00	2 KB
▼ emulated	drwxrwx---	2023-12-13 02:42	4 KB
▼ 0	drwxrws---	2023-12-13 02:42	4 KB
> Alarms	drwxrws---	2023-12-13 02:42	4 KB
▼ Android	drwxrws--x	2023-12-13 02:42	4 KB
> data	drwxrws--x	2025-10-20 02:16	4 KB
> media	drwxrws--x	2023-12-13 02:42	4 KB
> obb	drwxrws--x	2023-12-13 02:43	4 KB
> Audiobooks	drwxrws---	2023-12-13 02:42	4 KB
> DCIM	drwxrws---	2023-12-13 02:42	4 KB
> Documents	drwxrws---	2023-12-13 02:42	4 KB
> Download	drwxrws---	2023-12-13 02:42	4 KB
> Movies	drwxrws---	2023-12-13 02:42	4 KB
> Music	drwxrws---	2023-12-13 02:42	4 KB
> Notifications	drwxrws---	2023-12-13 02:42	4 KB



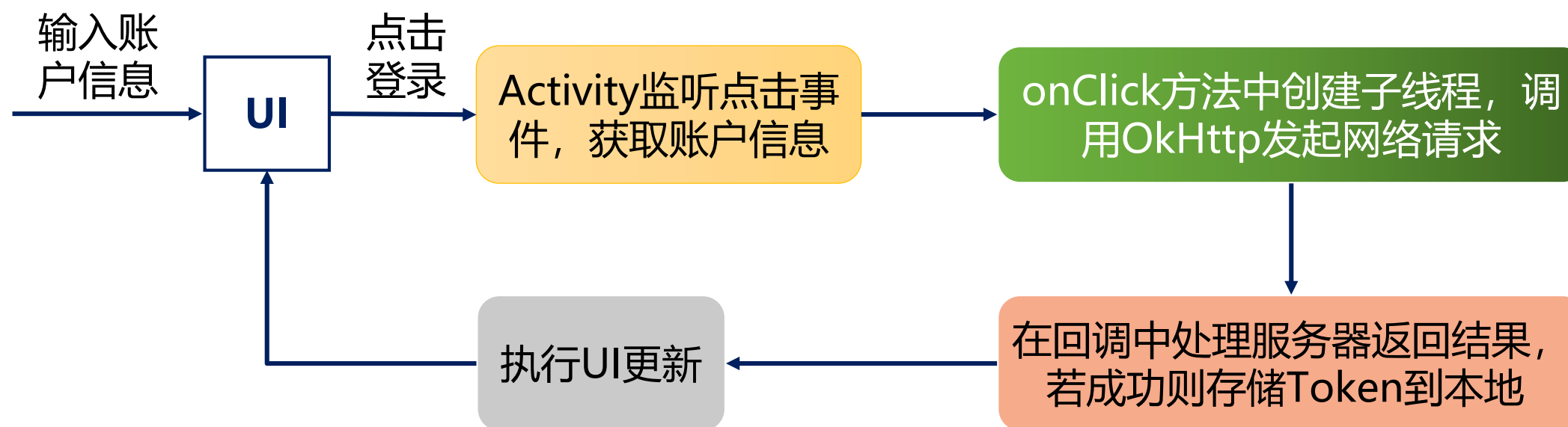
- **应用私有存储**: 其他应用无法直接访问，应用卸载时，系统自动删除。
- **公共存储**: 对其他应用和用户可见，应用卸载后不会被删除，Android 10之后要通过MediaStore (媒体文件) 或者SAF (普通文件) API访问。

Activity生命周期

- Android四大组件之一，负责将Android “页面” 中的按钮、文本框等UI控件展示在屏幕上，同时控制用户界面和与用户交互的逻辑。

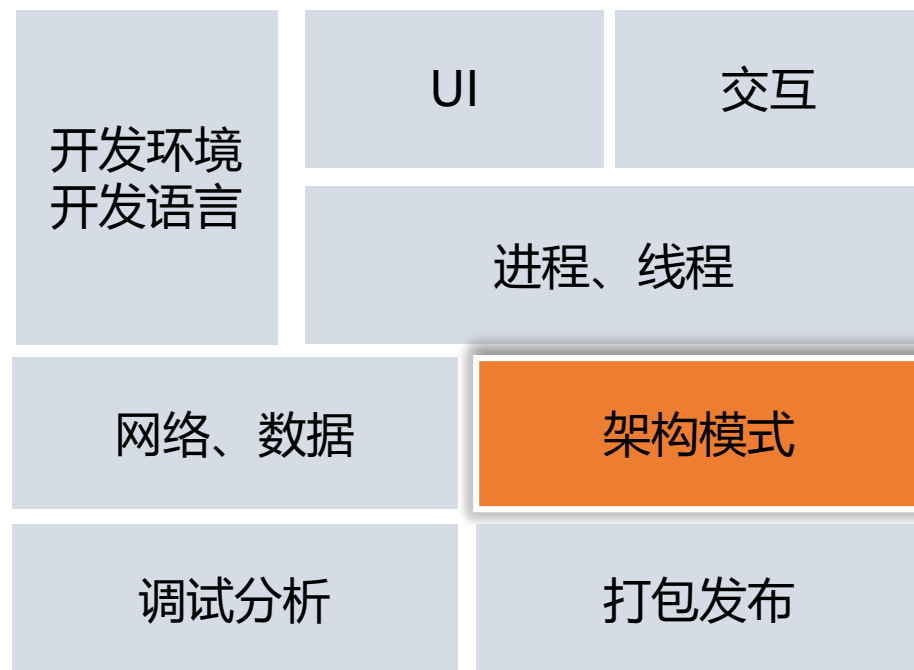


使用Activity类实现登录功能

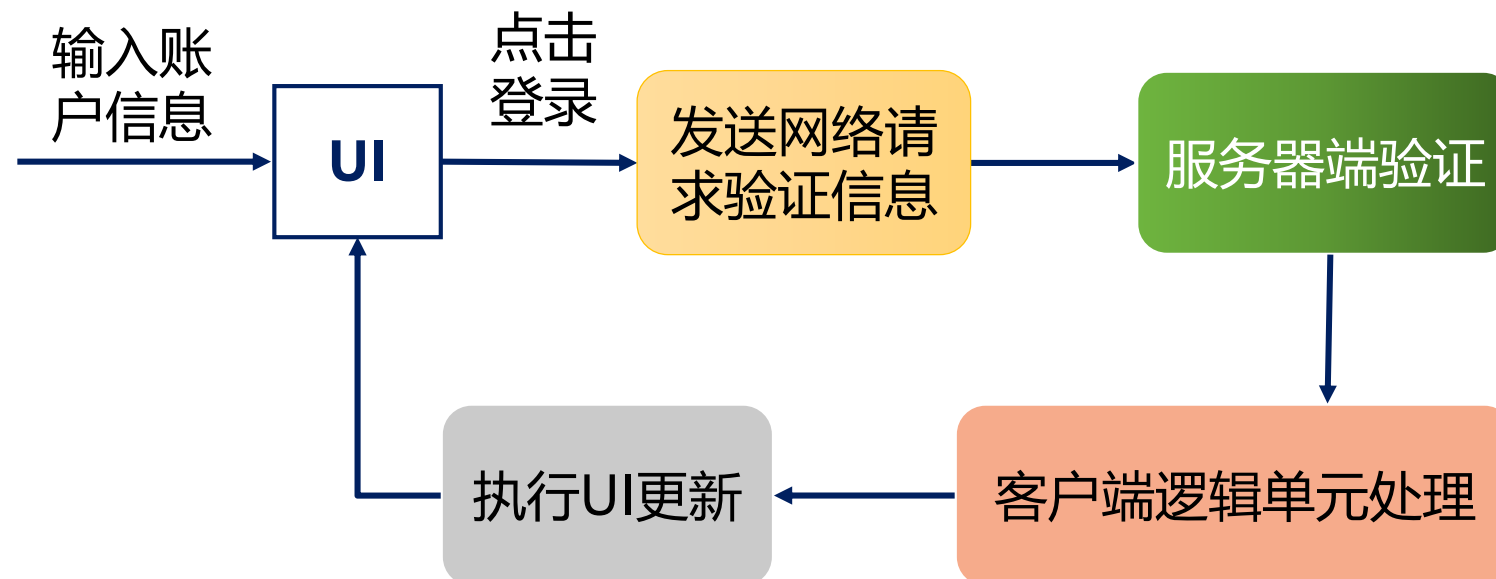


□ 问题

- ① Activity类代码混乱臃肿, 修改登录逻辑易出错
- ② 无法单独测试登录逻辑, 需要依赖Activity的生命周期



场景：登录



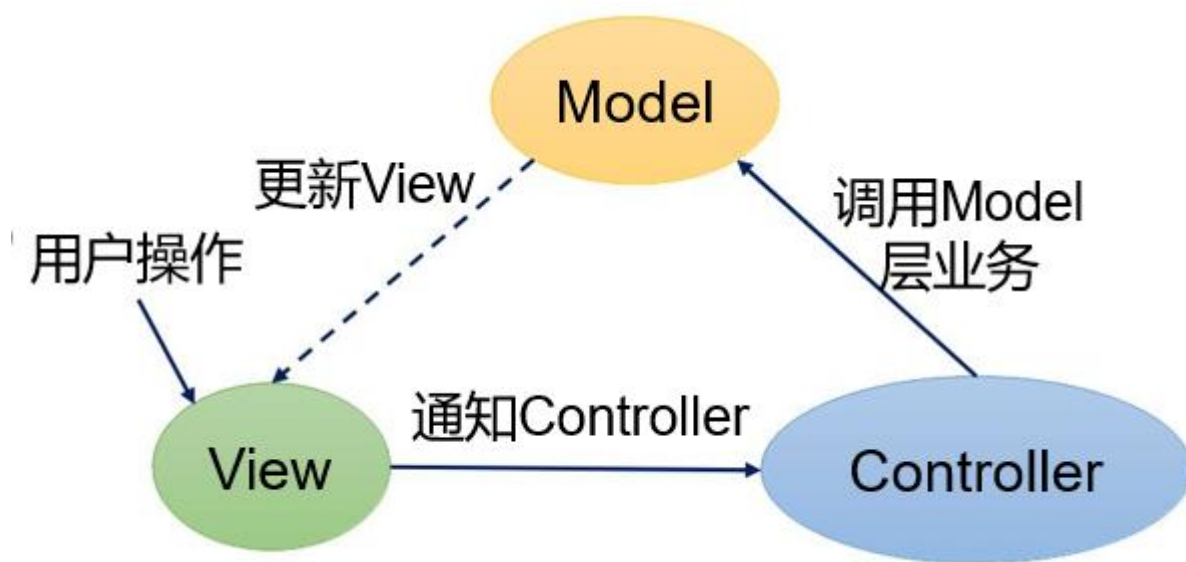
Android应用常见的开发架构模式有MVC、MVP和MVVM;

手段：通过软件分层、接口隔离等;

目的：降低耦合，减少迭代过程的改动，增强稳定性，减少项目维护成本。

MVC: (模型(Model) - 视图(View) - 控制器(Controller))

一种业务逻辑、数据、界面显示分离的代码组织方法。



View: UI视图层，显示Model层的数据结果。

Model: 模型层，处理业务逻辑数据（网络请求、数据库存取、算法操作等）。

Controller: 在Android中一般为Activity/Fragment，获取View层的数据，并向Model层发起请求。

MVC流程:

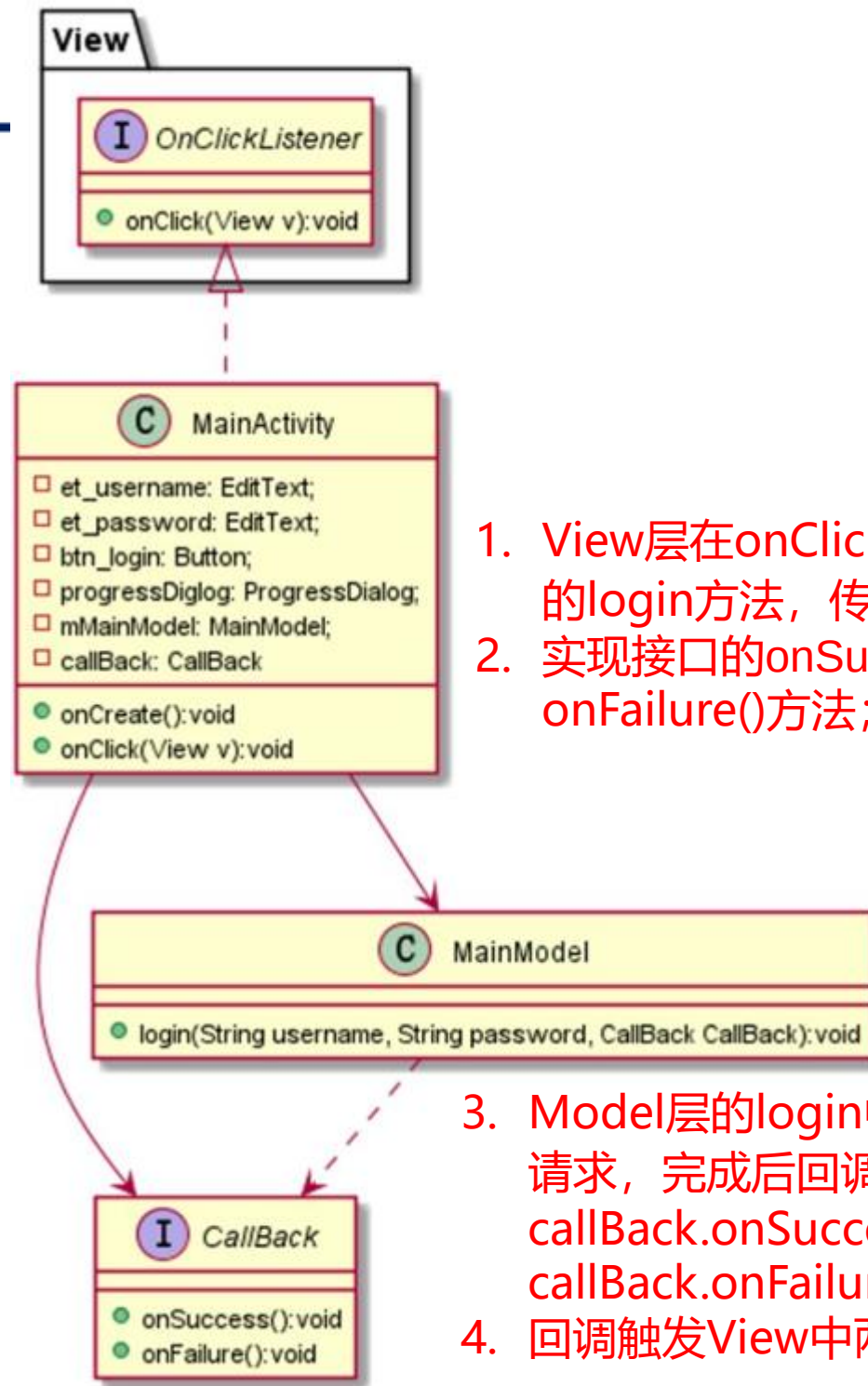
- View接受用户的操作
- View将用户的操作交给Controller
- Controller向Model发起请求
- Model数据处理后通知View更新

□ 优点:

简单易上手，一定程度降低了代码间的耦合性；

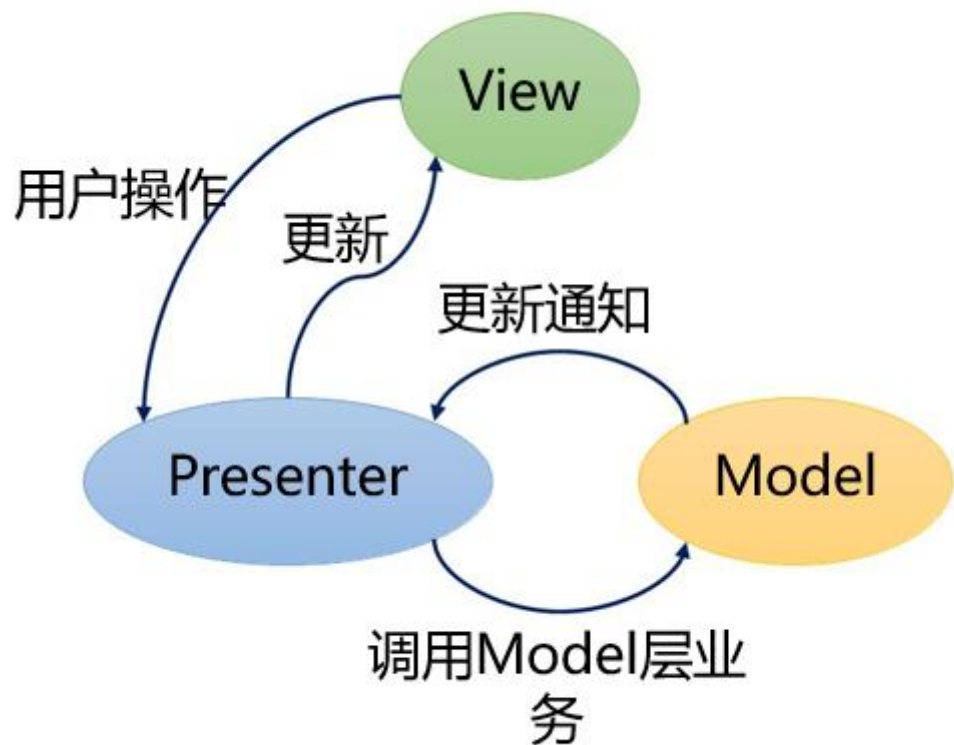
□ 缺点:

- Activity既有View的性质，也具有Controller的性质，导致这个类的职责不断增加，以致变得庞大臃肿。
- View会与Model直接交互，所以Activity与Model的耦合性很高。



1. View层在onClick方法中调用model的login方法，传入Callback接口
2. 实现接口的onSuccess()和onFailure()方法；
3. Model层的login中处理实际的网络请求，完成后回调callback.onSuccess() 和 callback.onFailure()；
4. 回调触发View中两个接口方法执行

MVP: Model (模型层) -View (视图层) -Presenter (协调器/主持者)

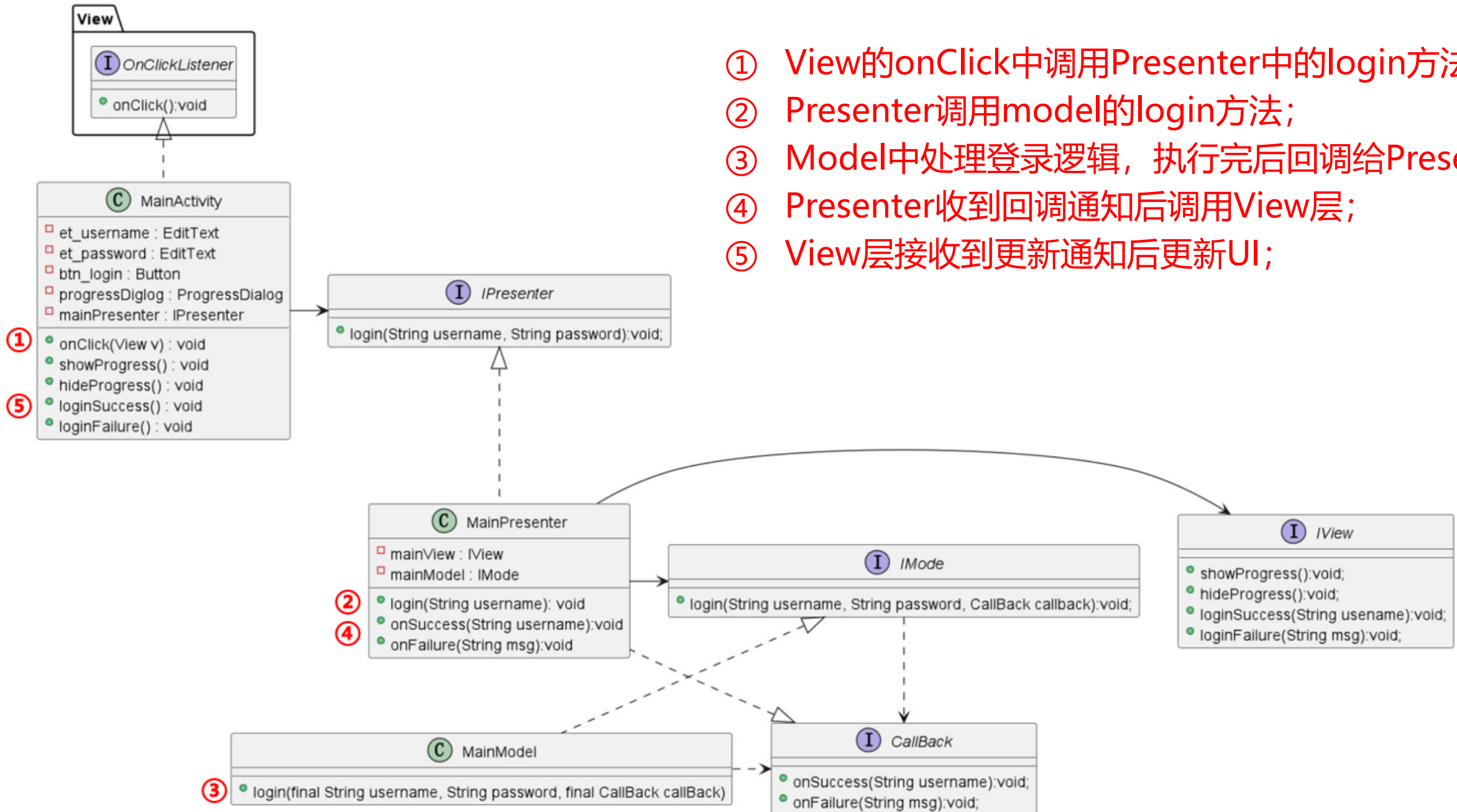


MVP优点:

- ①Model层与View层完全分离;
- ②可以将大量的逻辑操作放到Presenter中, 避免Activity的臃肿;

缺点:

- ①MVP模式View和Presenter双向依赖, 解耦依然不够彻底。
- ②需要定义各层接口进行实现, 接受Presenter回调的地方必须实现对应的接口。
- ③需要实现多余的模板方法, 新增很多类, 并且可读性不好。



- ① View的onClick中调用Presenter中的login方法;
- ② Presenter调用model的login方法;
- ③ Model中处理登录逻辑, 执行完后回调给Presenter;
- ④ Presenter收到回调通知后调用View层;
- ⑤ View层接收到更新通知后更新UI;

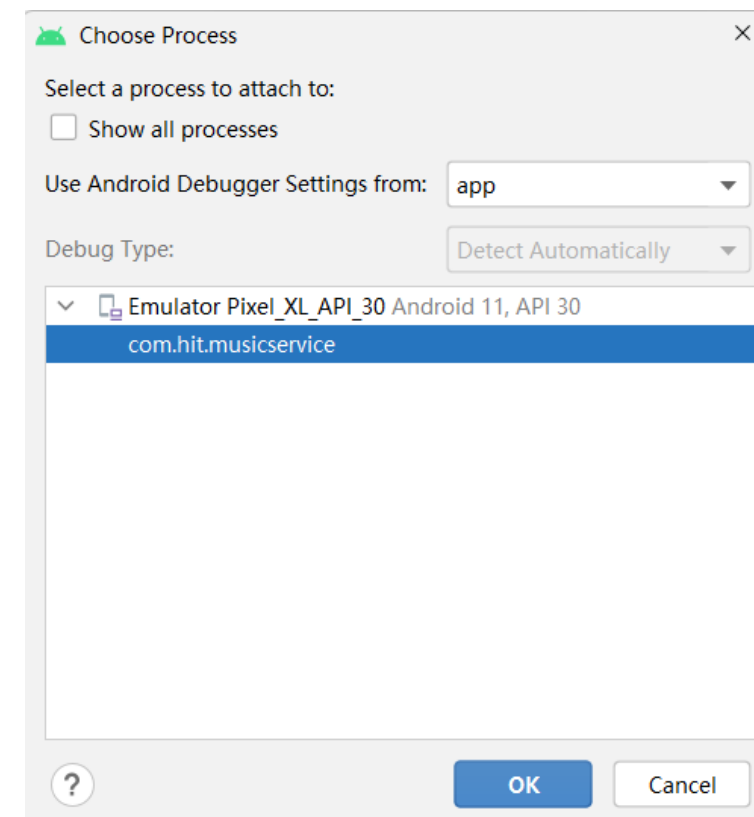
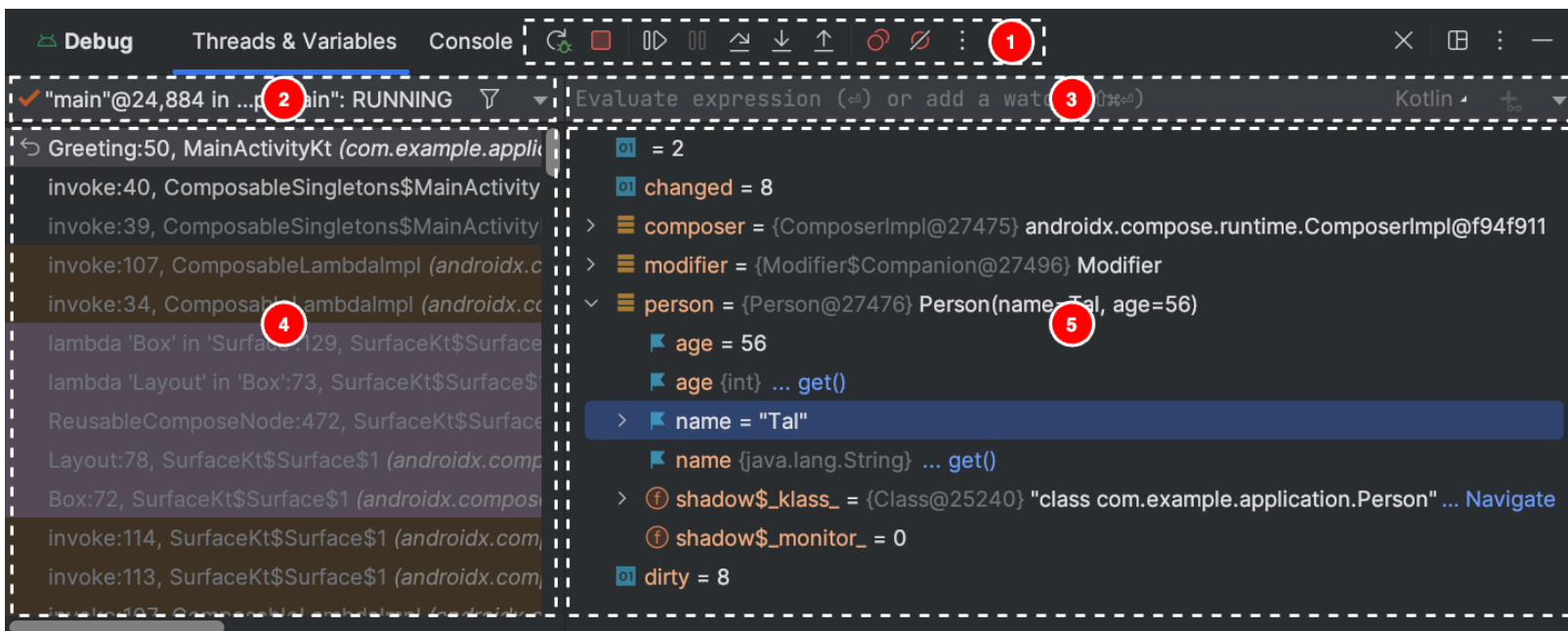


□ Android Studio调试器

□ Android系统日志

Android Studio调试器

- 在有可能出问题的代码处设置断点。
- 单击工具 “Debug app” 按钮  开始调试
- “Debug app” 按钮会重启应用程序，需要等待较长时间。如果程序已经运行，可以点击 “Attach Debugger To Android Process” 按钮  会弹出一个选择进程的对话框，选择项目进程。
- 使用调试工具，监视代码执行过程



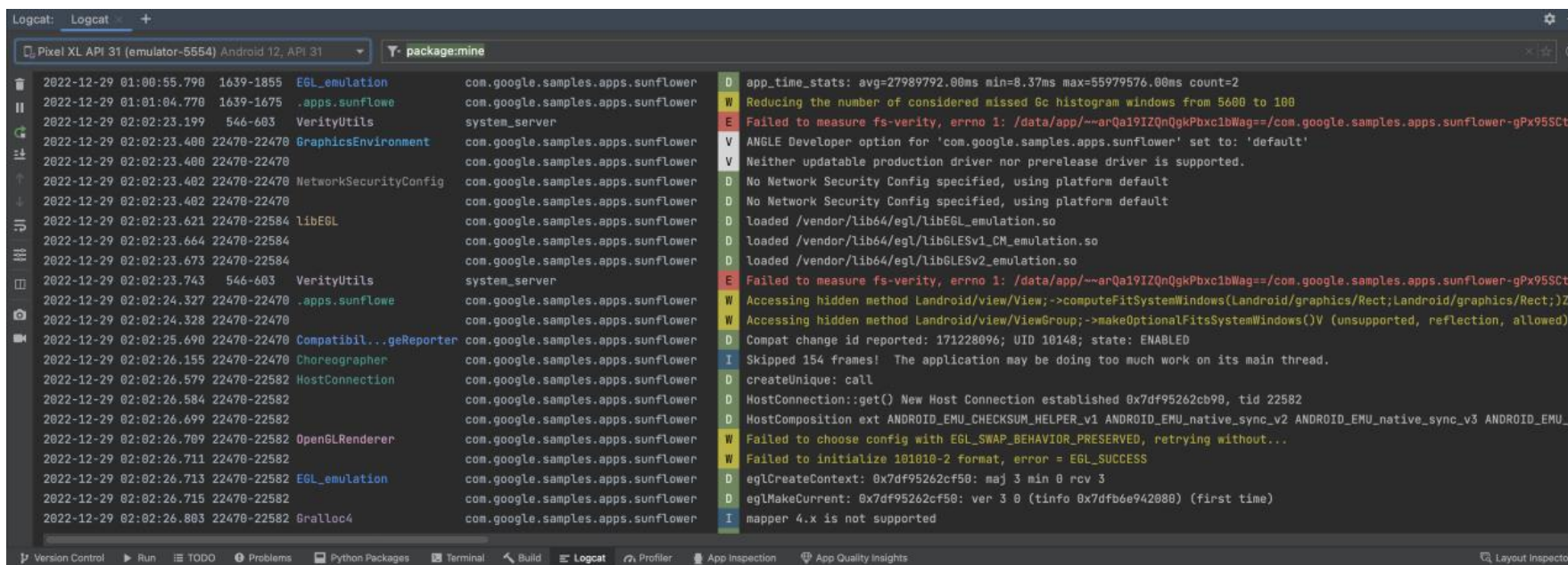
- ① 执行和导航工具栏
- ② 线程选择器
- ③ 求值和监视表达式条目
- ④ 堆栈显示
- ⑤ 变量窗格

- 在代码中使用**Log类**写入日志消息，日志消息会输出到Android 系统的调试输出流，从而帮助了解执行流程

优先级: Verbose < Debug < Info < Warn < Error

级别	作用	对应方法
Verbose	用于记录最详细的辅助信息，通常仅在精细调试时使用	Log.v(String tag, String msg)
Debug	用于开发阶段的详细调试信息，帮助定位具体问题，如循环变量的值	Log.d(String tag, String msg)
Info	用于记录程序的关键执行节点，反映正常的业务流程，帮助开发者了解程序运行状态，如用户登陆成功。	Log.i(String tag, String msg)
Warn	表示存在潜在问题，不会直接导致程序崩溃，但可能影响功能正常运行或性能，如使用了不推荐的 API。	Log.w(String tag, String msg)
Error	用于记录程序运行中出现的严重错误,通常会 导致功能异常或崩溃，如空指针异常	Log.e(String tag, String msg)

- Android日志显示在Logcat窗口中，从菜单栏中选择View->Tool Windows -> Logcat



The screenshot shows the Logcat window in Android Studio, displaying a list of log messages. The logs are organized into columns: Time, Level, Tag, Package, and Message. The messages include system-level logs (e.g., EGL_emulation, GraphicsEnvironment, NetworkSecurityConfig) and application-level logs (e.g., .apps.sunflower, VerityUtils, system_server). The interface includes a search bar at the top and a toolbar at the bottom with icons for Version Control, Run, TODO, Problems, Python Packages, Terminal, Build, Logcat, Profiler, App Inspection, and App Quality Insights.

- 每个日志都有一个日期、时间戳、进程和线程ID、标记Tag、软件包名称、优先级和消息
- 使用键值对搜索查询日志，如level: DEBUG

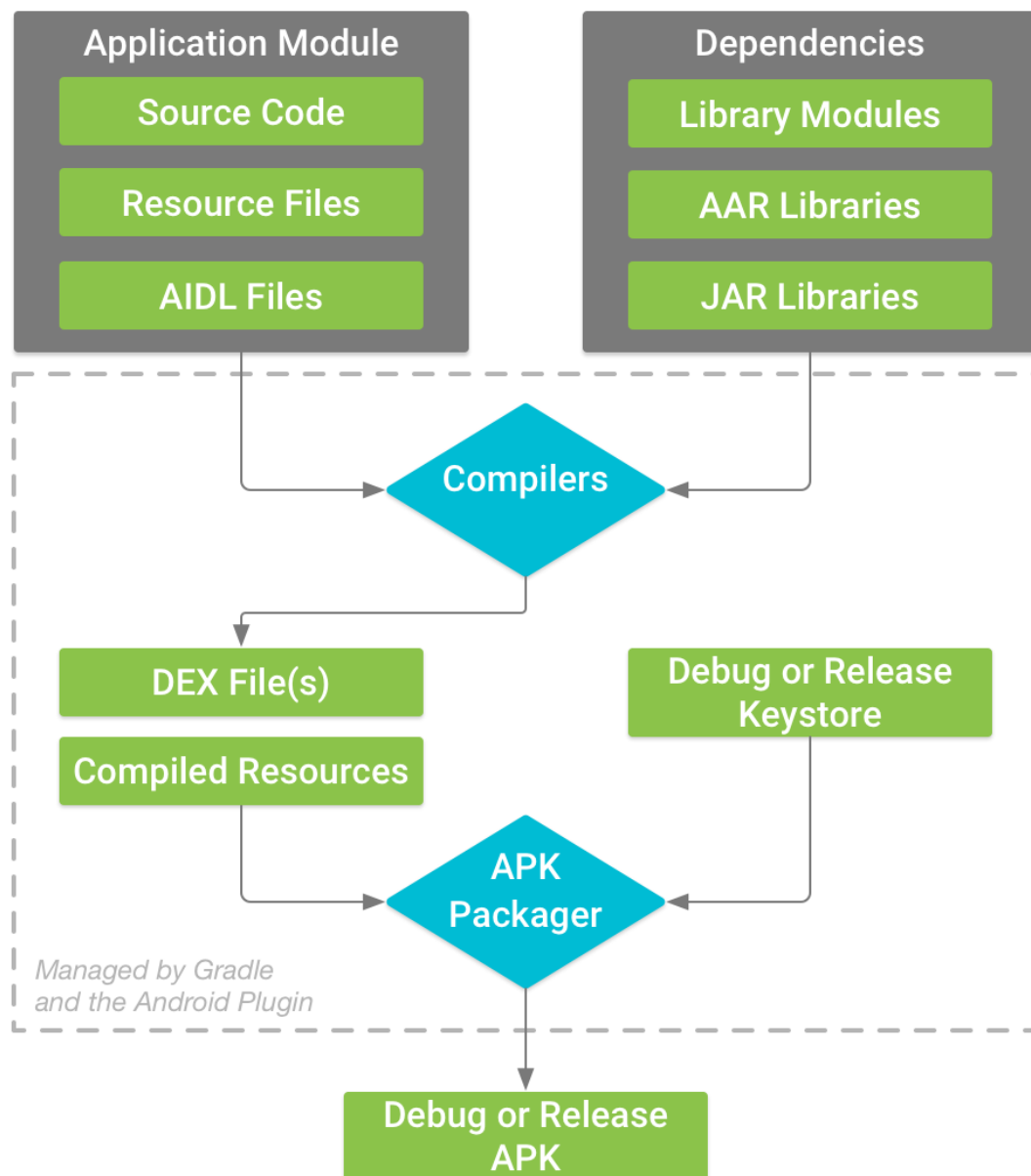


□ 构建调试版本包

作用：本地调试，打印调试日志，排查问题，验证功能

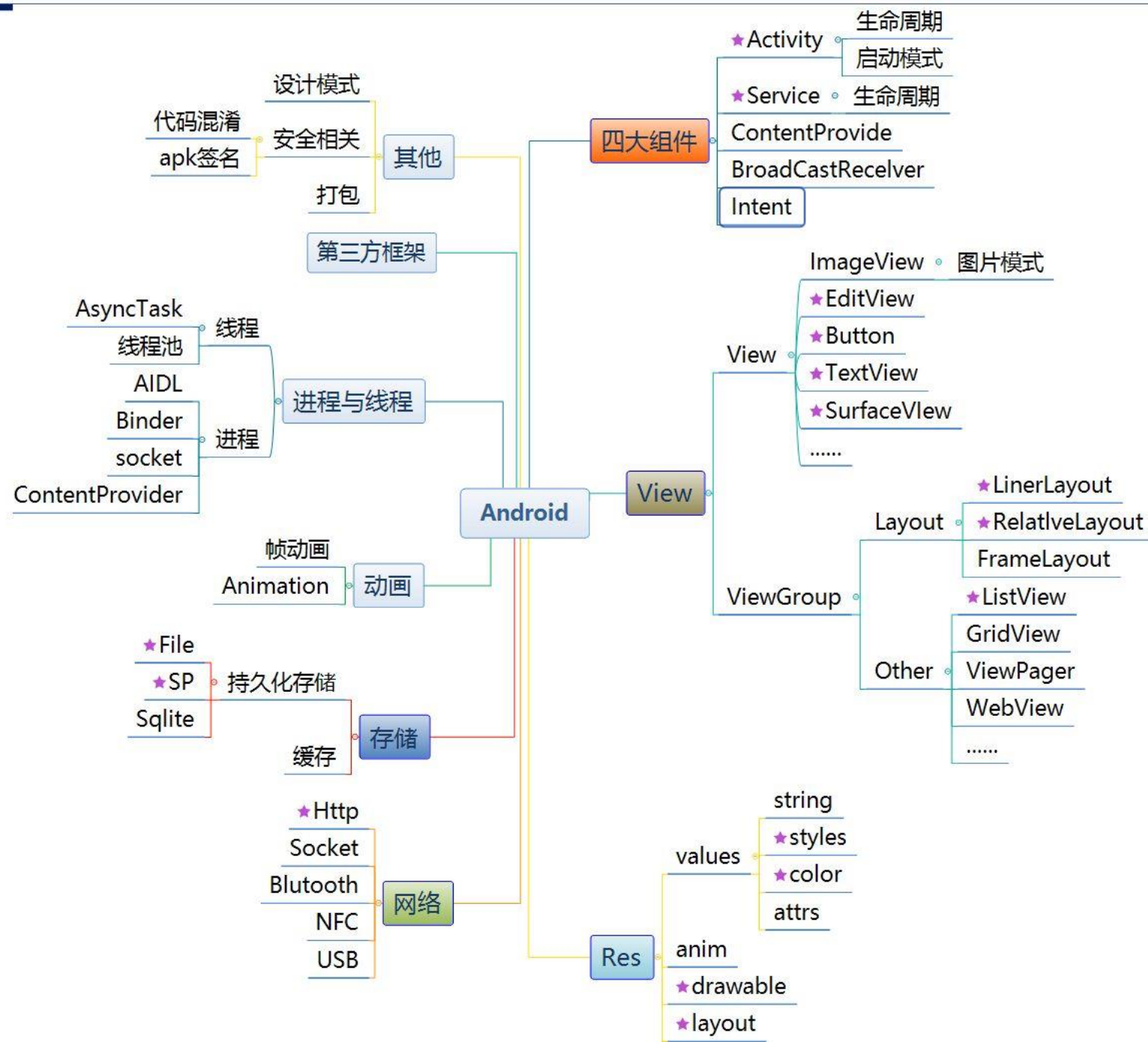
□ 构建发布版本包

作用：发布到应用市场



典型Android应用模块的构建流程

- ① 编译器将源代码转换成**DEX文件**
- ② 打包器将**DEX文件**和编译后的**资源**组合成**未签名APK**
- ③ 打包器会使用[zipalign](#)工具对应用进行**优化**, 以减少其在设备上运行时所占用的内存
- ④ 签名工具使用调试密钥库或发布密钥库为**APK签名**, 生成最终APK



1. 认识Android系统平台架构
2. 掌握Android开发基本流程
3. 学习Android开发基础知识（UI、交互、架构模式、网络访问、数据存储、进程线程、调试分析、打包发布）
4. 开始Android开发实践

1. 每组1-2人，自行组队，填写共享文档
2. 以组为单位提交任务书
3. 任务书内容
 - **项目概述**：简要描述项目内容，并说明实施意义
 - **原有功能迁移分析**：选择迁移哪位小组成员的代码，列出待迁移的windows端代码中**可复用的部分**及需**重新适配安卓平台**的功能模块，并分别说明原因
 - **新增网络功能设计**：至少规划**2项网络功能**，分别说明功能内容、设计原因及简要技术实现思路
 - **小组人员分工及依据**
 - **预期成果**



2026春-软件构造实...

群号: 970726848



扫一扫二维码，加入群聊

谢谢！



HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HIT